



汇编语言与微机接口技术

第3章 指令系统（一）

2025年2月10日



信息与软件工程学院
School of Information and Software Engineering

主要内容

- ① 指令系统的一般概念
- ② 对操作数的寻址方式
- ③ **8086指令系统**

3.1

● 指令系统的一般概念



3.1 概述

- 01. 指令及指令系统
- 02. 指令的格式
- 03. 指令中的操作数类型
- 04. 指令字长与机器字长
- 05. 指令的执行时间

3.1 指令系统的一般概念



» 指令与指令系统

指令

控制计算机完成某种操作的命令。

指令系统

处理器所能识别的所有指令的集合。

指令的兼容性

同一系列机的指令都是兼容的。

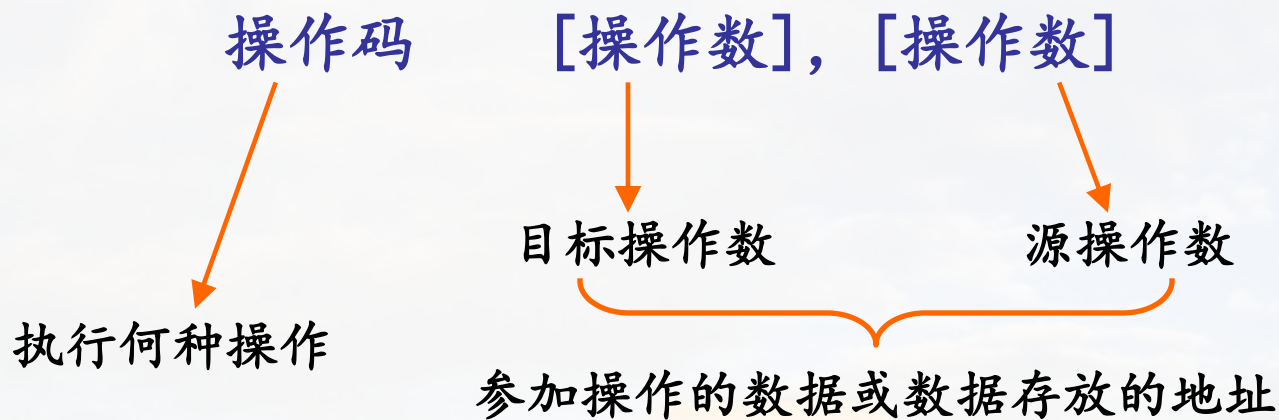
3.1 指令系统的一般概念

» 指令格式



3.1 指令系统的一般概念

指令的一般格式

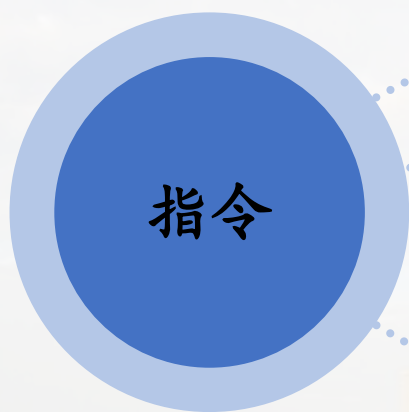


3.1 指令系统的一般概念

指令格式

根据操作数的数量，指令可以有以下格式

操作数：0，即不需
操作数，或隐含



零操作数指令： 操作码

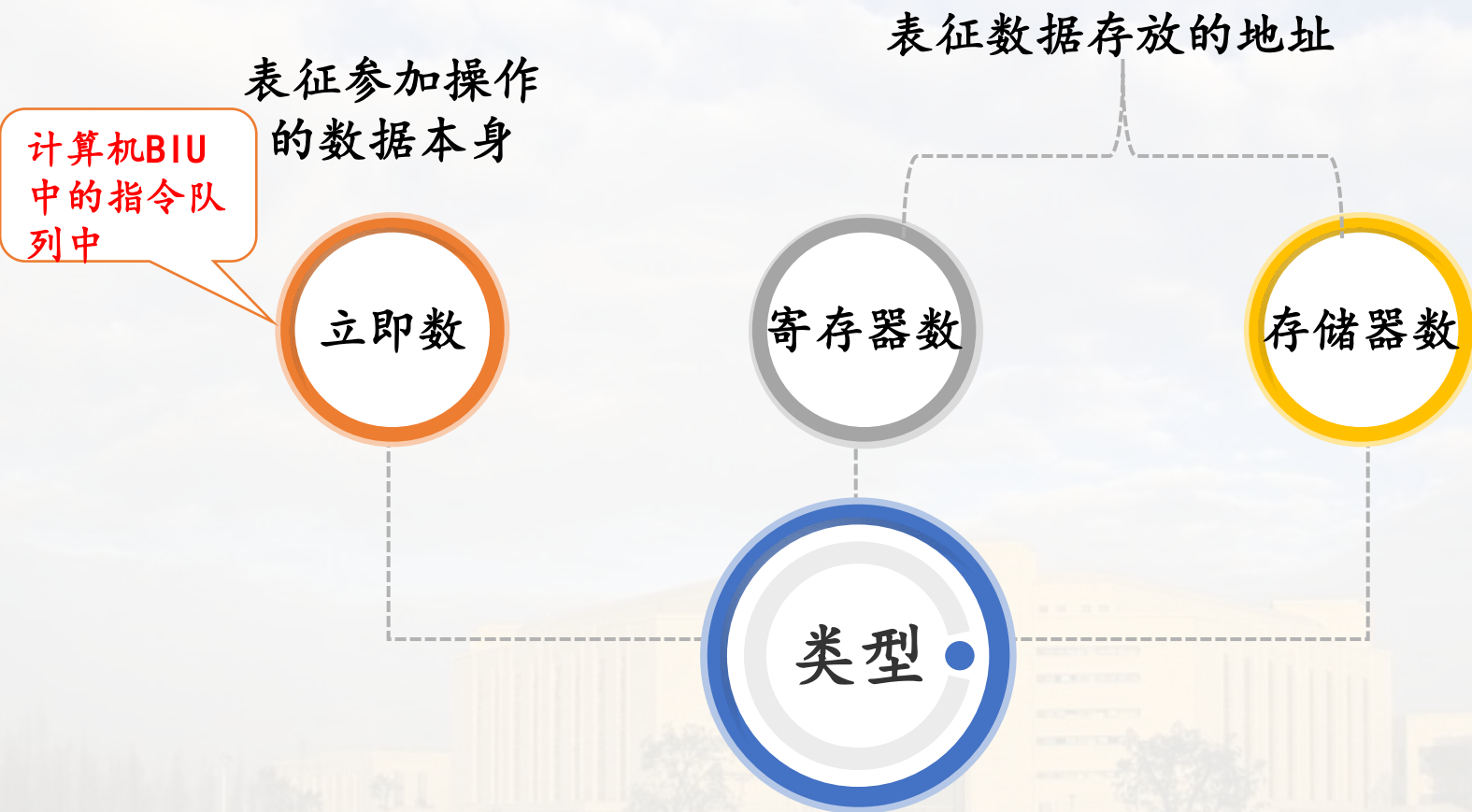
单操作数指令： 操作码 操作数

双操作数指令： 操作码 操作数，操作数

多操作数指令： 操作码 三操作数及以上

3.1 指令系统的一般概念

指令中的操作数类型



3.1 指令系统的一般概念

» 立即数操作数

▶ **立即数**本身是参加操作的数据，可以是8位或16位，**只能作为源操作数**。

例：MOV AX, **1234H** ;16位立即数

MOV BL, **22H** ; 8位立即数

立即数是运算数据本身，无地址含义，故无法作为目标操作数

3.1 指令系统的一般概念

» 寄存器操作数

- 参加运算的**数**存放在指令给出的**寄存器**中，可以是**16位**或**8位**。
- **寄存器**：**8个通用寄存器**，4个段寄存器（用得较少）。
- 例：
 - **MOV AX, BX**
 - **MOV DL, CH**

3.1 指令系统的一般概念

» 存储器操作数

只能是逻辑地址（偏移地址），而不能是物理地址

- 参加运算的数存放在存储器的某个单元中
- 表现形式：[]

寄存器或常数形式

寻找存储器操作数的关键是确定数据在内存中的存放地址

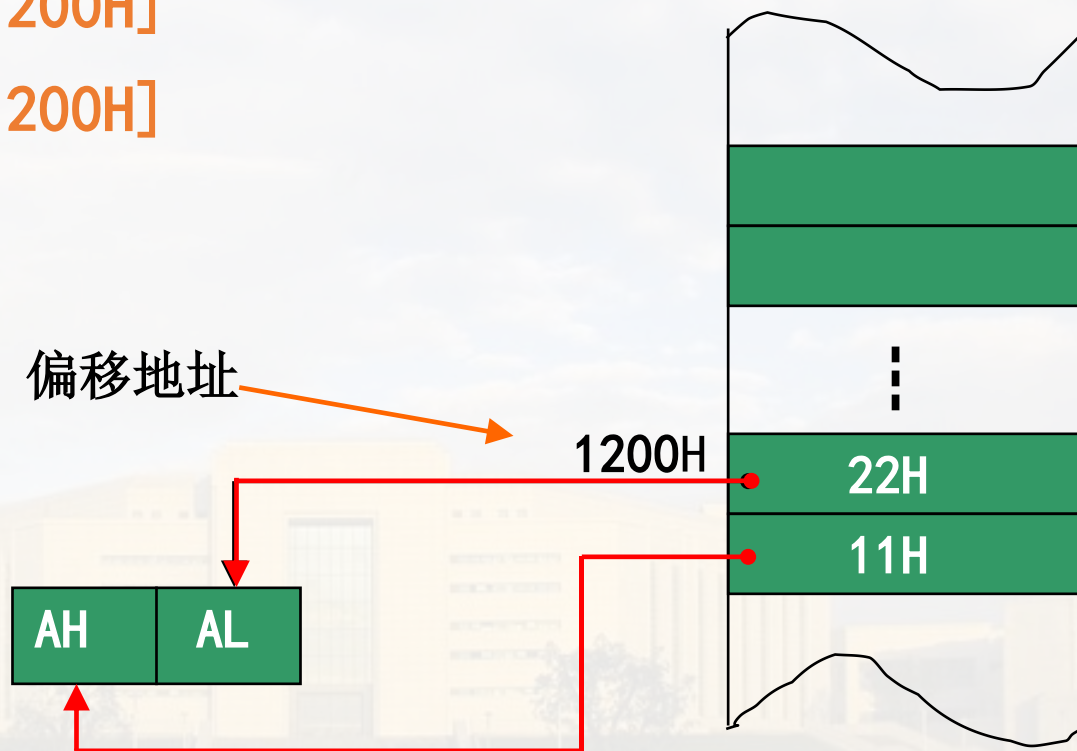
3.1 指令系统的一般概念

» 存储器操作数例

■ 例：

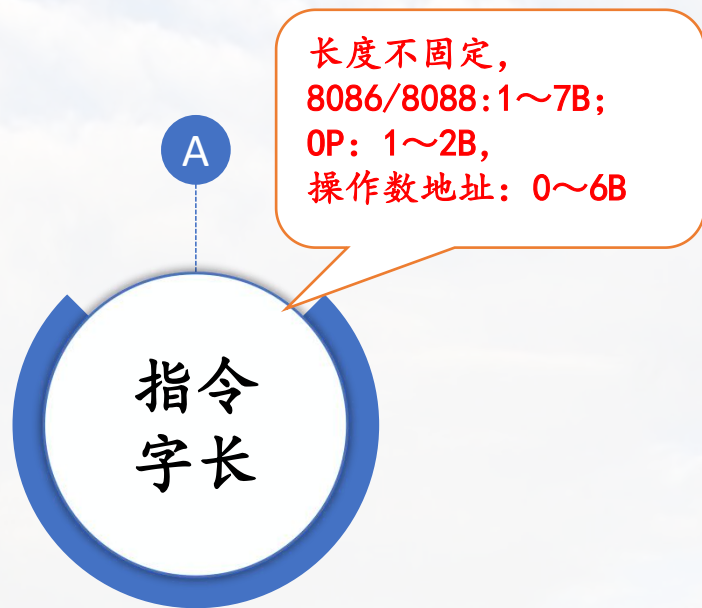
■ **MOV AL, [1200H]**

■ **MOV AX, [1200H]**



3.1 指令系统的一般概念

指令字长与机器字长



由操作码的长度、
操作数地址长度、
操作数个数决定;



计算机中ALU能够一
次直接处理的二进制
数的位数。

3.1 指令系统的一般概念

指令的执行速度

常用指令主要是1~4B



指令的字长影响指令的执行速度

(8086/8088常用指令的执行时间见附录C.1)



对不同的操作数，指令执行的时间不同：



3.2

寻址方式

3.2 寻址方式

» 寻址方式



» 寻址方式



▶ 操作数可能的来源或存放处

- 由指令直接给出：立即数
- 寄存器：寄存器数
- 内存单元：存储器数
- I/O接口：存储器数



▶ 寻找操作数所在地址的方法可以有三种大类型

- 指令直接给出的方式
- 存放于寄存器中的寻址方式
- 存放于存储器中的寻址方式（难点、重点）
- （含I/O接口的端口地址）

3.2 寻址方式

8086/8088对段存储器使用的约定

序号	内存访问类型	默认段寄存器	可重设的段寄存器	段内偏移地址来源
1	取指令	CS	无	IP
2	堆栈操作	SS	无	SP
3	串操作之源串	DS	ES、SS	SI
4	串操作之目标串	ES	无	DI
5	BP用作基址寻址	SS	ES、SS	按寻址方式计算得有效地址EA
6	一般数据存取	DS	ES、SS	按寻址方式计算得有效地址EA

有效地址EA即《计算机组成原理》课程中的操作数地址

《微机原理》课程中的操作数地址（物理地址）=段基址+段内偏移地址
(或EA)

EA: effective address

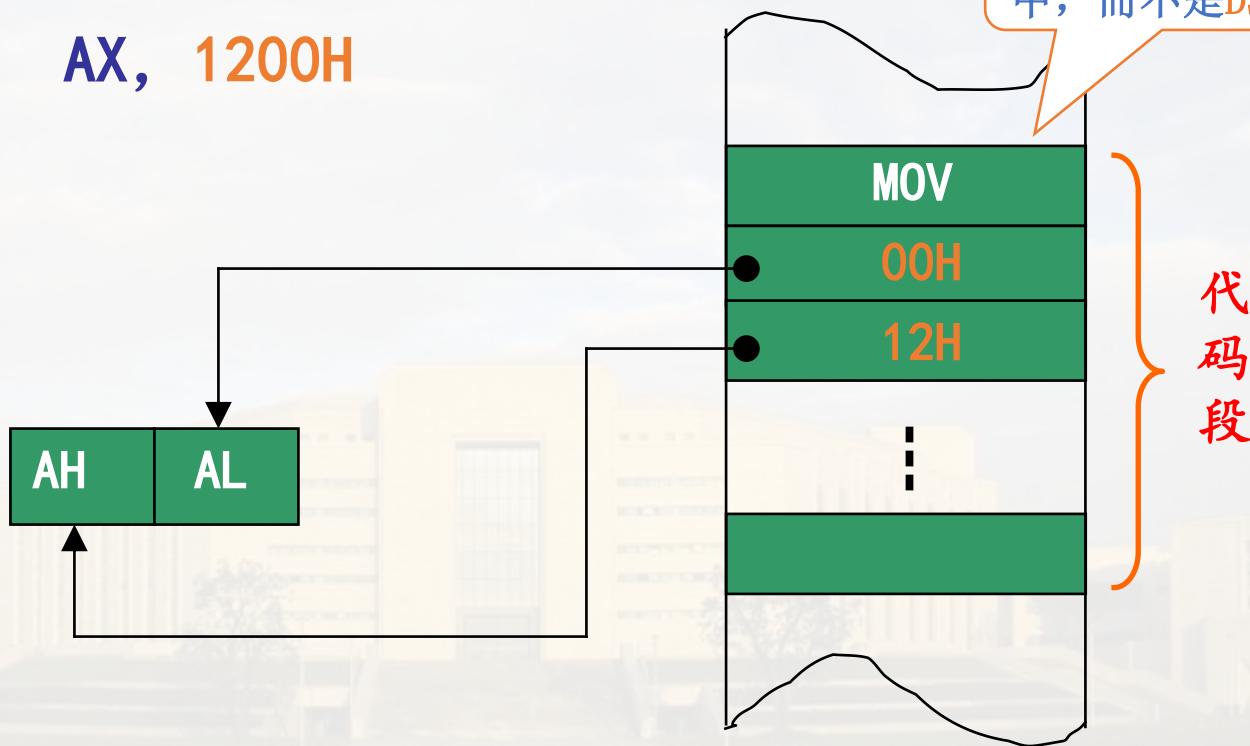
3.2 寻址方式

立即寻址

立即寻址仅适合于
源操作数

- 指令中的源操作数是立即数，即源操作数是参加操作的数据本身
- 例：MOV AX, 1200H

立即数一定放在CS
中，而不是DS中



» 寄存器寻址

- 操作数在CPU的8个通用寄存器或4个段寄存器（用得很少）中。

- 例：MOV AX, BX



3.2 寻址方式

直接寻址

- 指令中直接给出操作数的偏移地址

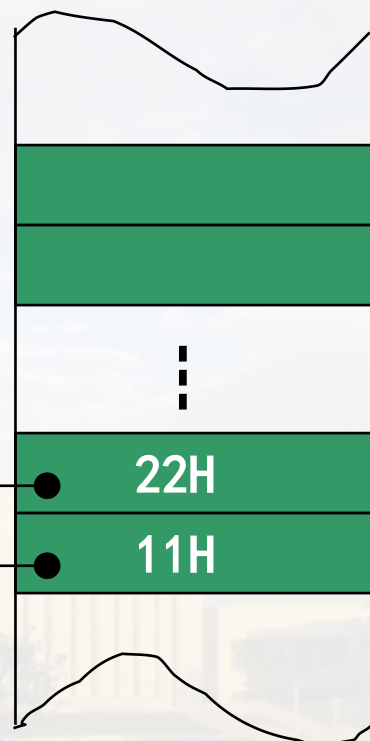
- 默认在数据段DS中

- 例：MOV AX, [1200H]

偏移地址

1200H

在数据段DS中



数据段

3.2 寻址方式

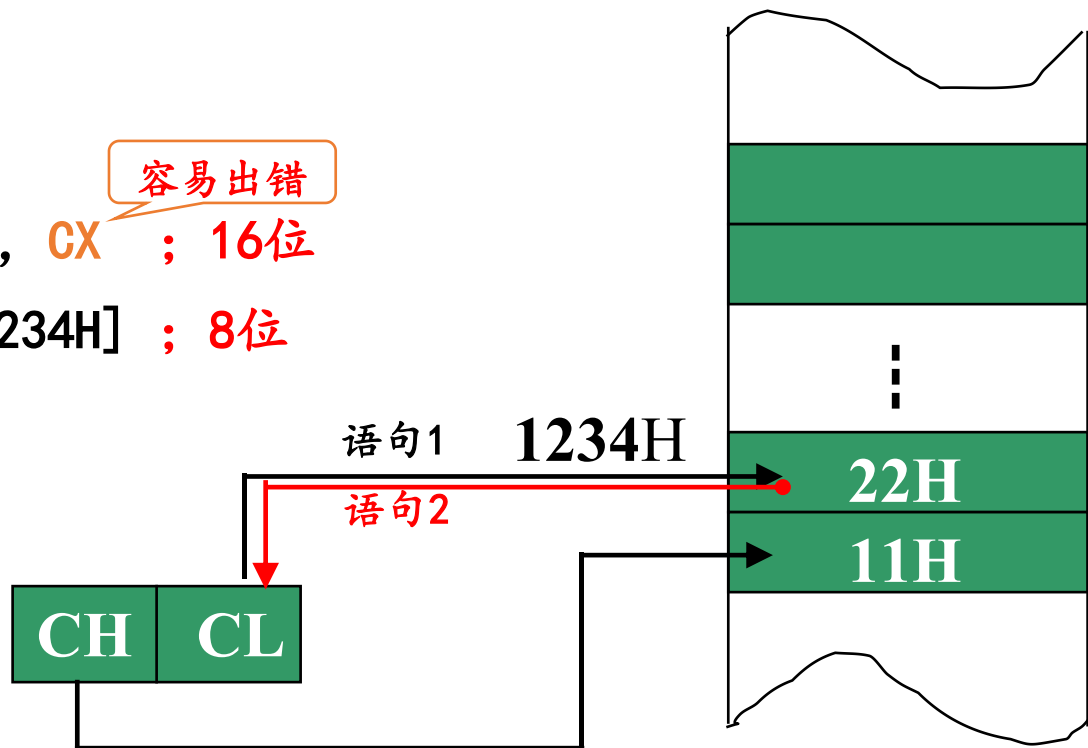
直接寻址

注意点：

- 存储器操作数的长度由指令中另一个操作数的长度决定。

例：

- MOV [1234H], CX ; 16位
- MOV CL, [1234H] ; 8位



» 直接寻址

- 注意点（续）：

- 直接寻址方式下，操作数的段地址默认为数据段，但允许段重设，即由指令定义段，说明数据存放在其它逻辑段中。

- 例：MOV AX, ES: [1200H]

- 这种情况称为段超越，所加的段寄存器叫段前缀。

» 寄存器间接寻址



参与操作的数据存放在内存中，其偏移地址为指令中寄存器的内容。

例：



- `MOV BX, 1200H`
- `MOV AX, [BX]`
- ；等价于 `MOV AX, DS:[BX]`

BX的值即为数据存放处的偏移地址



3.2 寻址方式

寄存器间接寻址

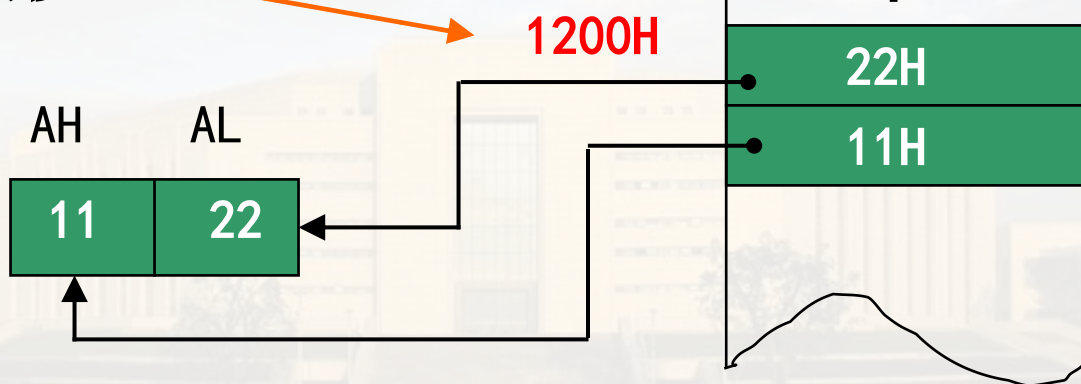
例：寄存器间接寻址示意图

MOV BX, 1200H

MOV AX, [BX]

；等价于 MOV AX, DS:[BX]

偏移地址



代码段

数据段

3.2 寻址方式

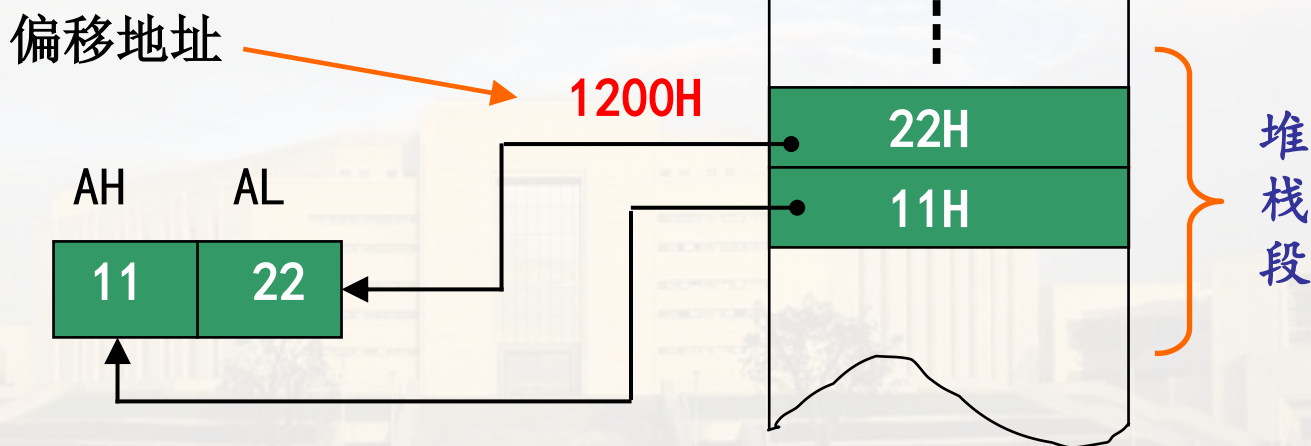
寄存器间接寻址

例：寄存器间接寻址示意图

MOV BP, 1200H

MOV AX, [BP]

；等价于 MOV AX, SS: [BP]



寄存器间接寻址

总结

01

由寄存器间接给出操作数的**段内偏移地址**；

02

存放偏移地址的寄存器称为**间址寄存器**，它们是以下四个之一：**BX, BP, SI, DI**

03

偏移地址长度均为16位，故间址寄存器必须是16位寄存器。

04

操作数的段地址（数据处于哪个段）取决于选择哪一个间址寄存器：

BX, SI, DI → 默认在**数据段 (DS)**

BP → 默认在**堆栈段 (SS)**

3.2 寻址方式

寄存器相对寻址: (组成原理的变址/基址寻址)



操作数的偏移地址为寄存器的内容加上一个8/16位位移量 (常量)

8位位移量: 短位移量
16位位移量: 长位移量



可以使用的寄存器为: BX、BP、SI和DI



格式例:

MOV AX, DATA[BX]
或: MOV AX, [BX+DATA]
或: MOV AX, [BX] DATA
或: MOV AX, [BX]+DATA

应用: 一维数组操作;
偏移地址: 起始地址为位移量,
元素下标放间址R: 如BX放下标



例:

MOV BX, 220H

MOV CL, 5H[BX]

DATA可以放在BX之前

3.2 寻址方式

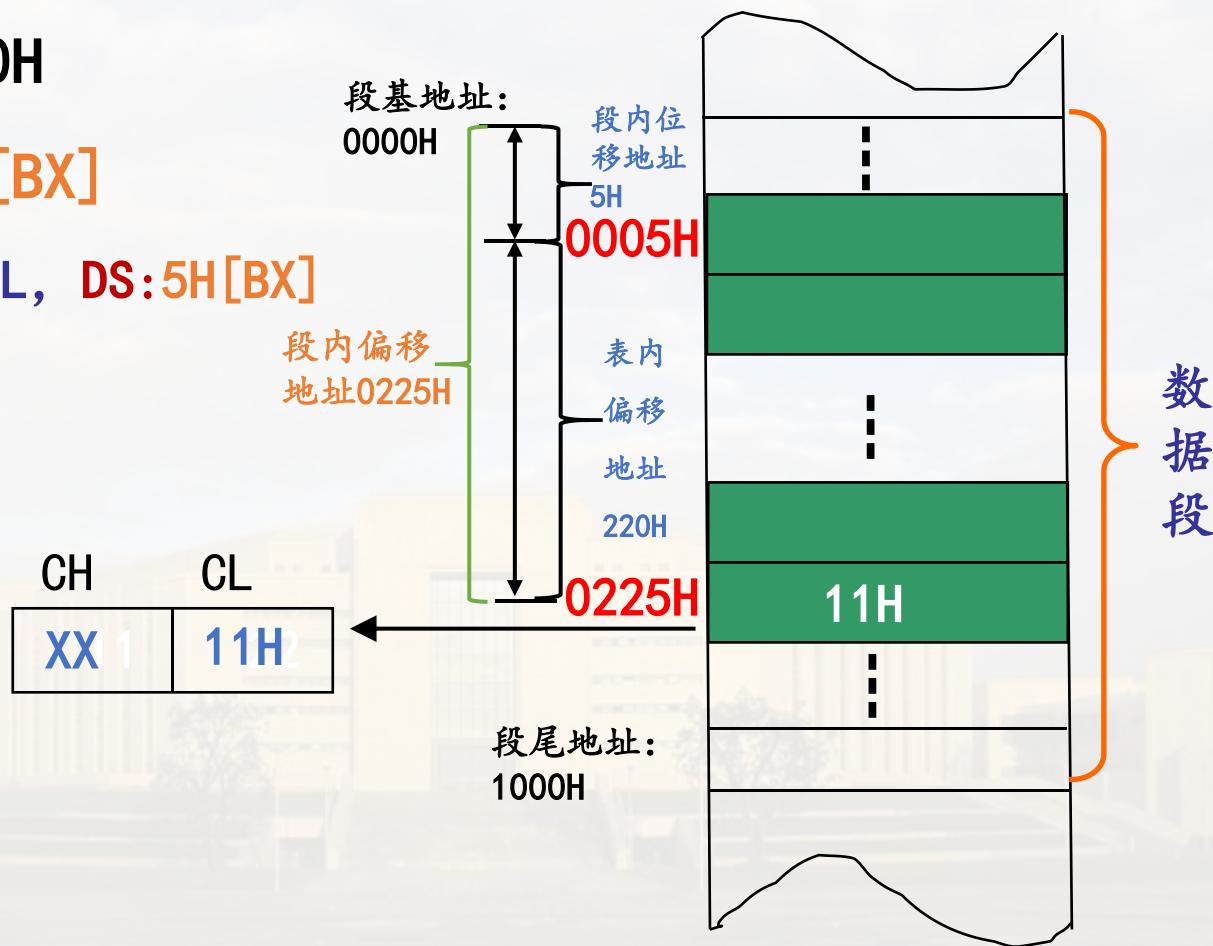
寄存器相对寻址: (组成原理的变址/基址寻址)

例: 寄存器相对寻址示意图

MOV BX, 220H

MOV CL, 5H[BX]

; 等价于 MOV CL, DS:5H[BX]



» 寄存器相对寻址总结

操作数的段内偏移地址为

- 一个变址/基址寄存器的内容 + 位移量的内容;

操作数的段地址由选择的基址寄存器决定

- 基址寄存器为BX，默认在数据段DS
- 基址寄存器为BP，默认在堆栈段SS

寄存器相对寻址方式，主要用于一维数组操作。

3.2 寻址方式

» 基址-变址寻址

操作数的段内偏移地址为

- 一个基址寄存器的内容 + 一个变址寄存器的内容;

操作数的段地址由选择的基址寄存器决定

- 基址寄存器为BX，默认在数据段DS
- 基址寄存器为BP，默认在堆栈段SS

基址变址寻址方式与寄存器相对寻址方式一样，主要用于一维数组操作。

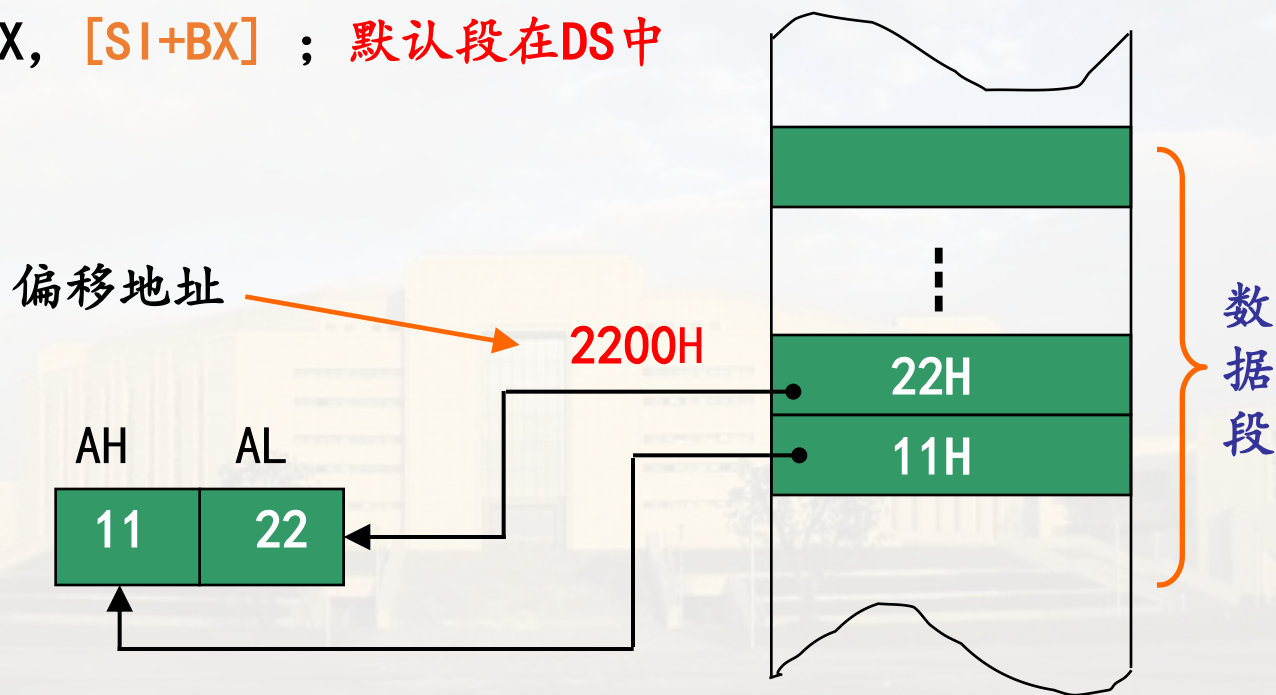
3.2 寻址方式

基址-变址寻址

例：

■ 执行下列程序段：

- MOV SI, 2000H
- MOV BX, 200H
- MOV AX, [SI+BX] ; 默认段在DS中



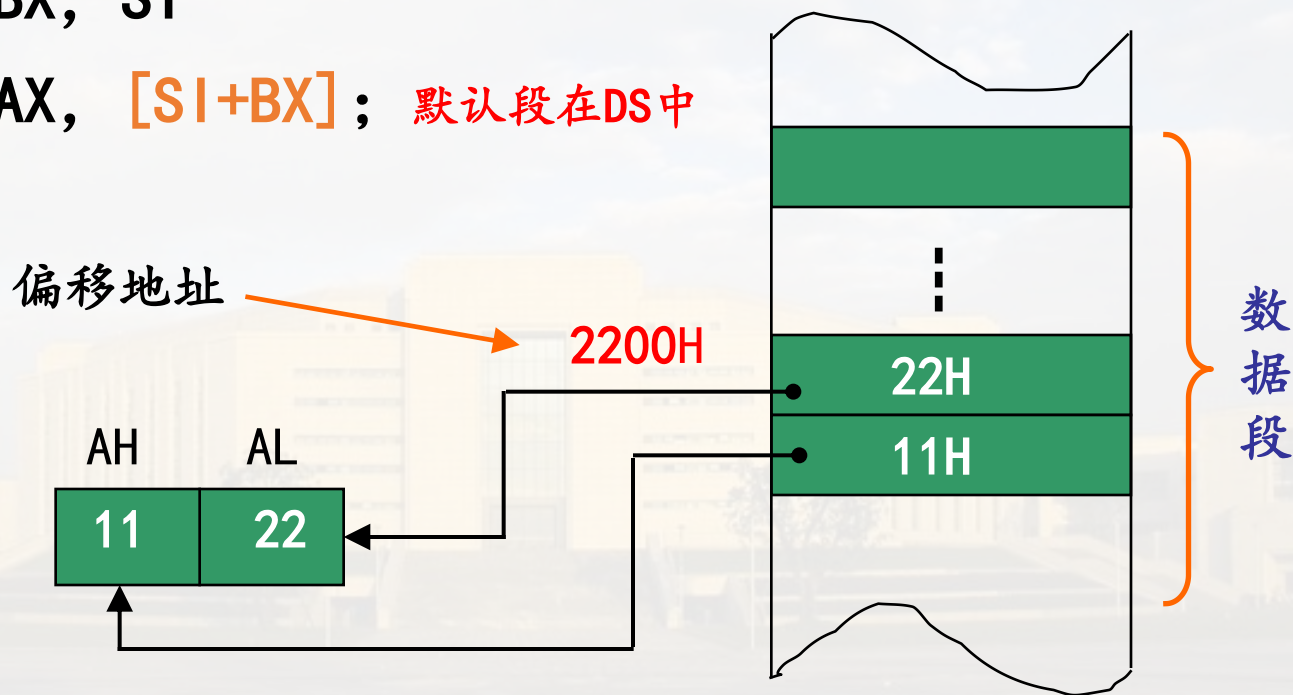
3.2 寻址方式

基址-变址寻址

例：

■ 执行下列程序段：

- MOV SI, 1100H
- MOV BX, SI
- MOV AX, [SI+BX] ; 默认段在DS中



3.2 寻址方式

» 基址-变址-相对寻址

操作数的段内偏移地址为

- 一个基址寄存器的内容 + 一个变址寄存器的内容+位移量;

操作数的段地址由选择的基址寄存器决定

- 基址寄存器为BX，默认在数据段DS
- 基址寄存器为BP，默认在堆栈段SS

基址变址相对寻址方式，主要用于二维数组操作。

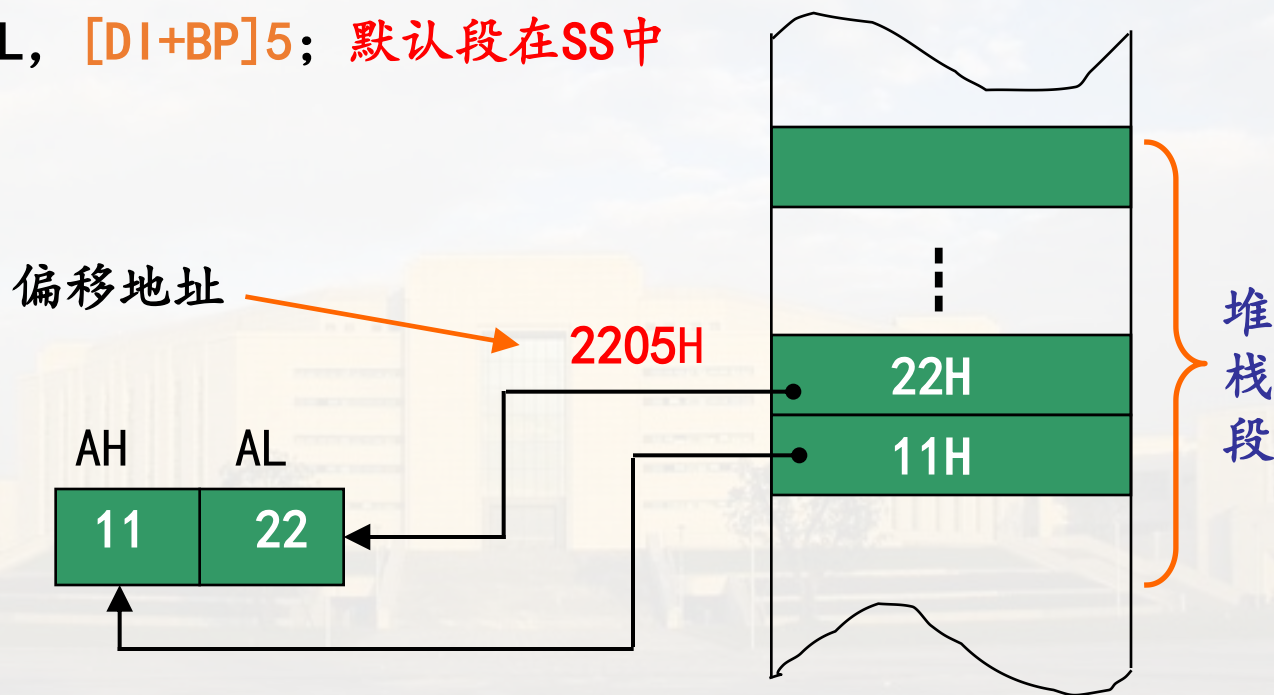
3.2 寻址方式

基址-变址-相对寻址

例：

■ 执行下列程序段：

- MOV DI, 1100H
- MOV BP, DI
- MOV AL, [DI+BP]5; 默认段在SS中



二、寻址方式

» 基址-变址-相对寻址

例：某商场的销售金额汇总表如表所示，采用基址加变址方式查询某天的销售额。

表 商场销售金额统计示意表

月 \ 日	金额 (万)							
		1	2		17		30	31
1		100	100		80		60	80
2		50	60		100		0	0
.....								
6		60	80		90		80	0
.....								
12		100	100		100		90	100

基址-变址-相对寻址方式查找6月17日的销售金额。

» 隐含寻址



指令中隐含了一个或两个操作数的地址，即操作数在默认的地址中。

• 例：



■ MUL BL

■ 隐含操作数AL

• 指令执行：

■ $AL \times BL \longrightarrow AX$

3.2 寻址方式

» 寻址方式中的几点注意事项

注意事项



立即寻址只适合于源操作数；

指令中的操作数若为[]形式，则表示操作数为存储器操作数。即：



- 要寻找的数据在内存中
- 该数据在内存中的存放地址为按寻址方式计算

使用寄存器来指示存储器单元的寄存器**只有4个**



- BX, SI, DI, BP
- 只有使用BP时默认的段为堆栈段，其它为数据段，可以使用段前缀来改变默认的段。

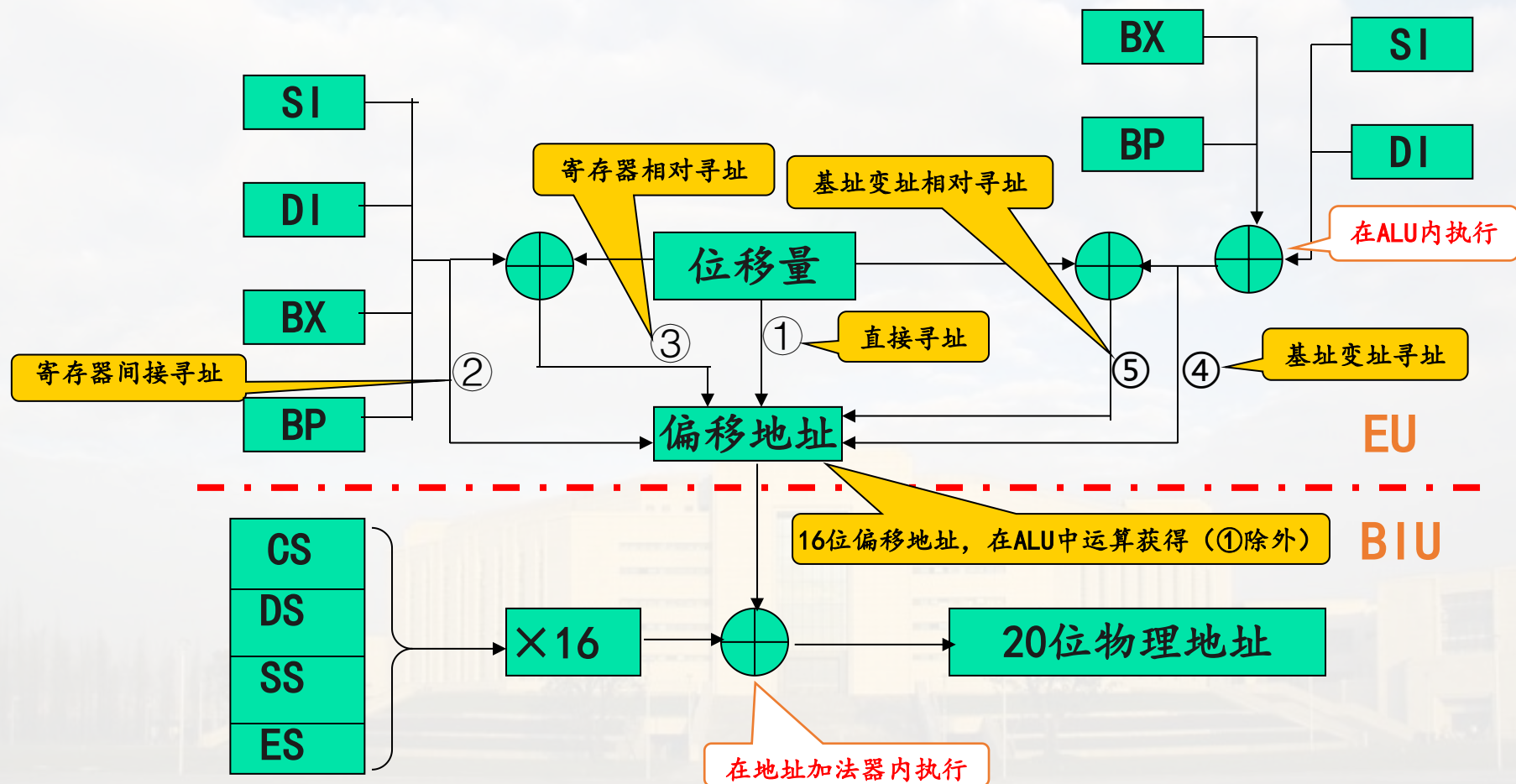
3.2 操作数的寻址方式（总结）



寻址方式	指令格式	可选寄存器	操作数默认段	操作数位置	备注
立即寻址	MOV AX, 1200H MOV AL, 12H			指令中，立即数	赋初值
寄存器寻址	MOV AX, BX MOV AL, BL	1) 16位通用寄存器AX、BX、CX、DX、BP、SP、SI、DI; 2) 8位通用寄存器 AH、AL、BH、BL、CH、CL、DH、DL; 3) 4个段寄存器CS、DS、ES、SS（用得很少）		寄存器中，寄存器数	送一个字 送一个字节
直接寻址	MOV AX, [1200H] MOV AL, [1200H]			存储器中，存储器数	送一个字 送一个字节
寄存器间接寻址	MOV AX, [BX] MOV AL, [BX]	只能使用4个寄存器BX、BP、SI、DI之一	BX、SI、DI → 默认在数据段DS中 BP → 默认在数据段SS中	存储器中，存储器数	送一个字 送一个字节
寄存器相对寻址 （变址、基址寻址）	MOV AX, DATA[BX] MOV AL, DATA[BX]	同上	同上	存储器中，存储器数	送一个字 送一个字节
基址-变址寻址	MOV AX, [BX][SI] MOV AL, [BX][SI]	只能使用2个寄存器BX、BP之一，加上2个寄存器SI、DI之一	同上	存储器中，存储器数	送一个字 送一个字节
基址-变址-相对寻址	MOVAX, DATA[BX][SI] MOV AL, DATA[BX][SI]	同上	同上	存储器中，存储器数	送一个字 送一个字节

3.2 寻址方式

➤ 存储器操作数寻址方式中地址形成小结：（总共8种，立即、寄存器、隐含寻址不在其内）



3.3

● 8086指令系统



3.3 掌握内容

- **01. 指令码的含义**

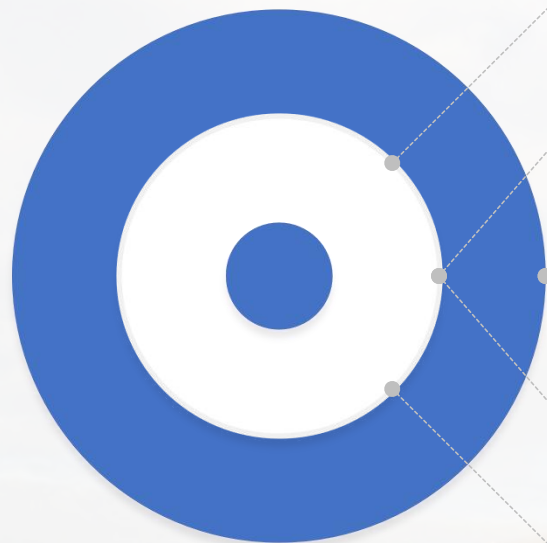
- **02. 指令对操作数的要求**

- **03. 指令对标志位的影响**

- **04. 指令的功能**

8086指令系统

从功能上包括六大类



数据传送



算术运算



逻辑运算和移位



串操作

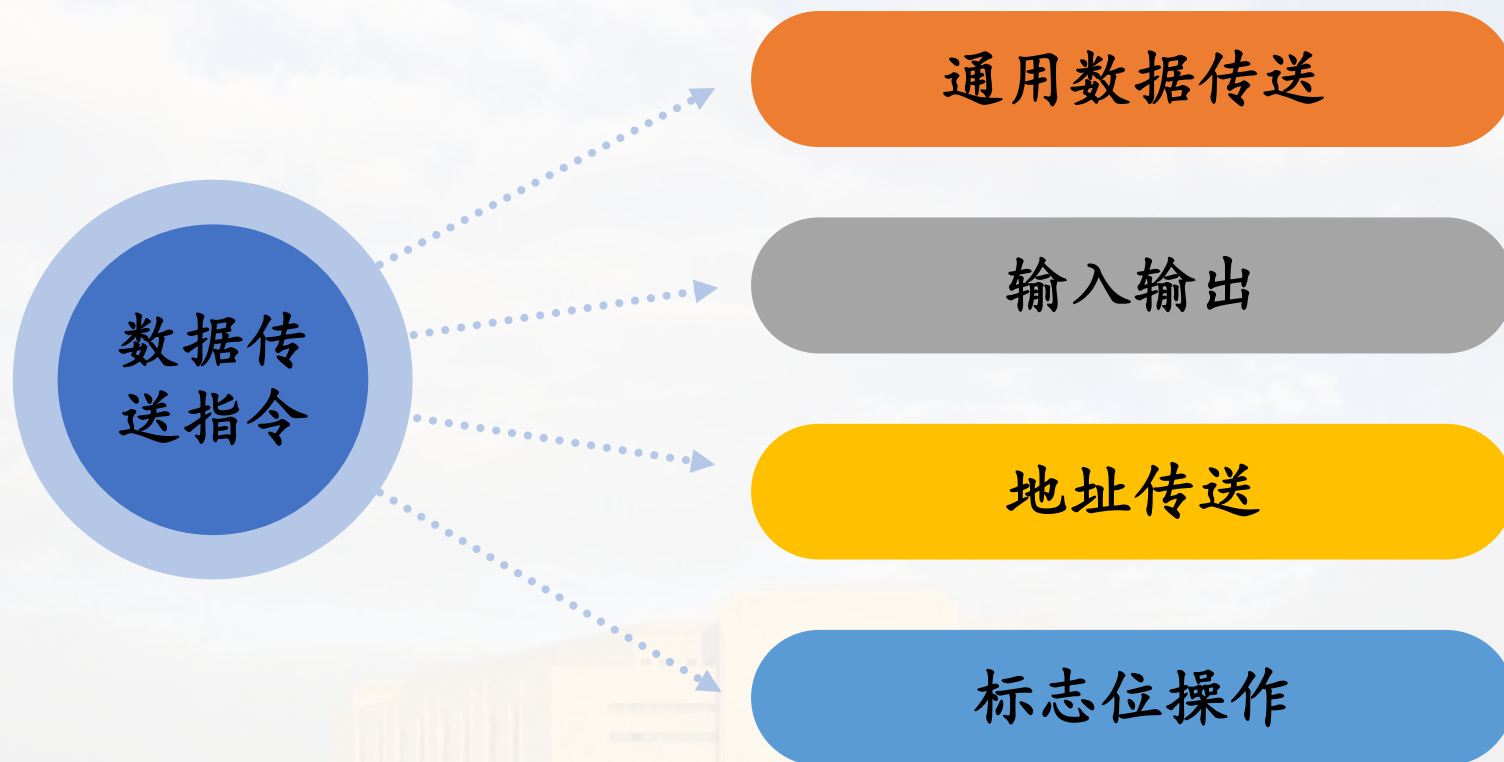


程序控制



处理器控制

» 数据传送指令



通用数据传送

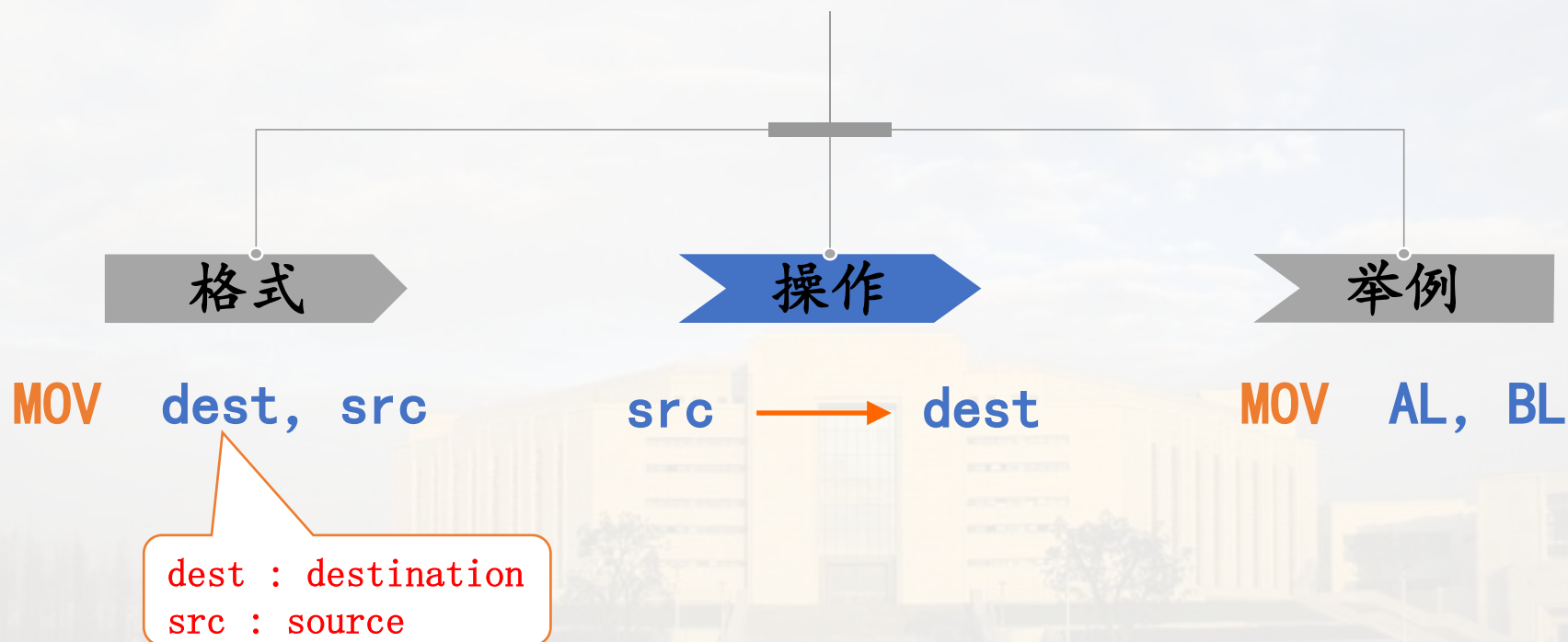


■ 特点：

- 该类指令的执行对标志位不产生影响

» 一般数据传送指令

一般数据传送指令 MOV



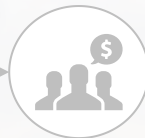
数据传送指令使用上的注意

注意



两操作数长度必须相同；

系统设计时，时序时间不够



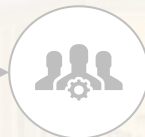
存储单元之间不能直接传送；



段寄存器CS只能作源操作数，段寄存器之间不能直接传送；



在源操作数是立即数时，目标操作数不能是段寄存器；



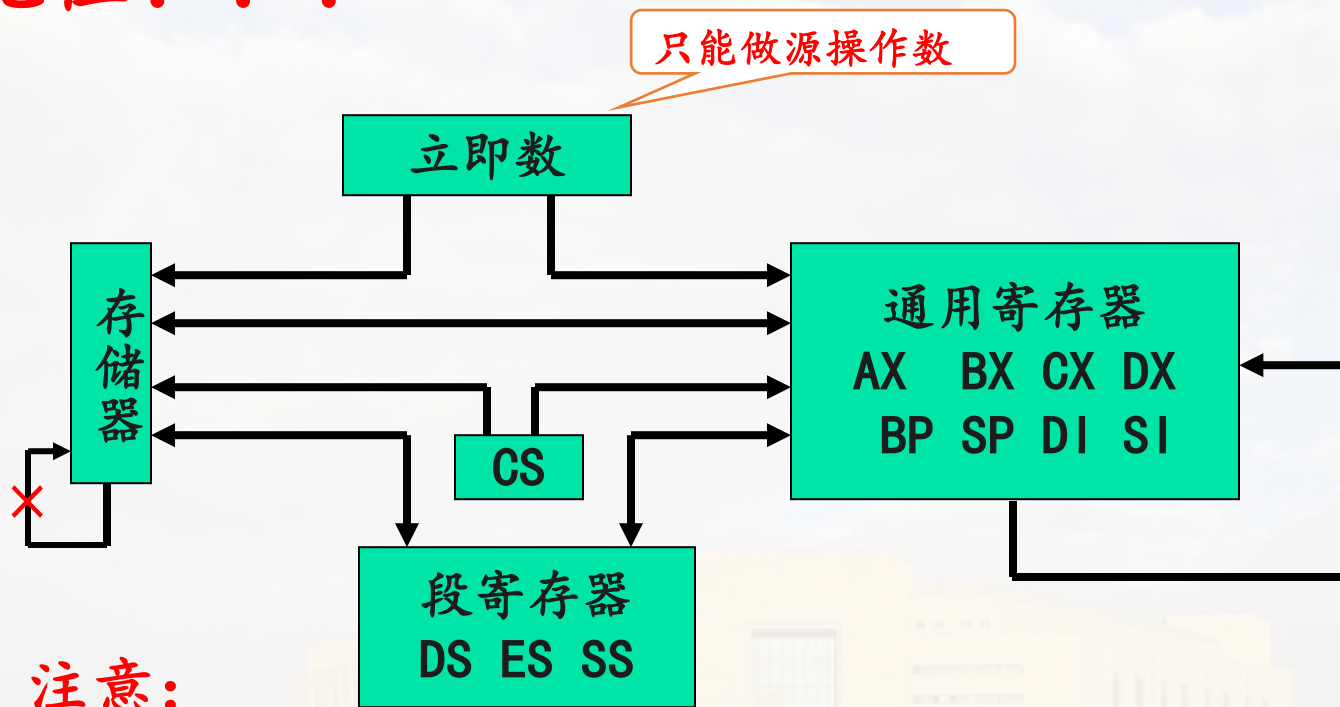
FLAGS寄存器一般不作为操作数在指令中出现。



MOV指令不影响标志位

数据传送指令的传送方向示意图

记住!!!



注意:

- 在《微机原理》两个存储器单元之间不能直接传送数据
- 在《计算机组成原理》两个存储器单元之间可以直接传送数据

» 一般数据传送指令例

判断下列指令的正确性：

- **MOV AL, BX**
- **MOV AX, [SI]05H**
- **MOV [BX][BP], BX**
- **MOV DS, 1000H**
- **MOV DX, 09H**
- **MOV [1200H], [SI]**

» 一般数据传送指令例

判断下列指令的正确性：

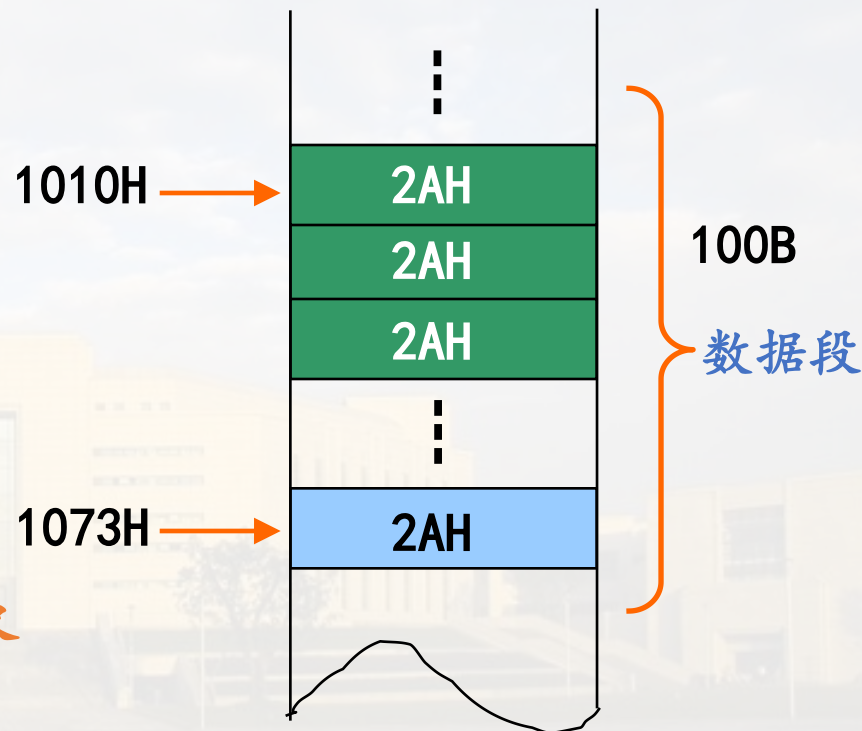
- | | |
|---------------------|------------------|
| ■ MOV AL, BX | × 长度不一 |
| ■ MOV AX, [SI]05H | ✓ |
| ■ MOV [BX][BP], BX | × BX , BP不能同时使用 |
| ■ MOV DS, 1000H | × 不能直接给段赋值 |
| ■ MOV DX, 09H | × 长度不一 |
| ■ MOV [1200H], [SI] | × 存储器单元之间不能直接传数据 |

➤ 一般数据传送指令应用例

- 将符号“*”的ASCII码2AH送入内存数据段1010H开始的100个单元中。

- 题目分析：

- 确定首地址
- 确定数据长度
- 写一次数据
- 修改单元地址
- 修改计数值
- 判断写完否？
- 未完继续写入，否则结束



» 一般数据传送指令应用例

程序段：

```
MOV    DI, 1010H
MOV    CX, 100
MOV    AL, ' *'
AGAIN: MOV [DI], AL
        INC    DI                ; DI+1
        DEC    CX                ; CX-1
        JNZ    AGAIN            ; CX≠0则继续
        HLT
```

» 上段程序在代码段中的存放形式

假设CS=109EH，IP=0100H，则各条指令在代码段中的存放地址如下：

CS : IP	机器指令	汇编指令
109E: 0100	B81010	MOV DI, 1010H
109E: 0103	.	MOV CX, 64H
109E: 0105	.	MOV AL, 2AH
109E: 0107	.	AGAIN: MOV [DI], AL
109E: 0109		INC DI
109E: 010A		DEC CX
109E: 010B		JNZ AGAIN
109E: 010D		HLT

» 数据段中的分布

送上2AH后数据段中相应存储单元的内容改变如下：

DS: 1010	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A
DS: 1020	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A
DS: 1030	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A
DS: 1040	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A
DS: 1050	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A
DS: 1060	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A	2A
DS: 1070	2A	2A	2A	2A	00	00	00	00	00	00	00	00	00	00	00	00

偏移地址[DI]

每行16个, 即1小节;
 $16 \times 6 + 4 = 100$

» 堆栈操作指令

有关堆栈的概念

- 栈顶、栈首、栈底

掌握

堆栈指令的操作原理

- 执行过程，执行结果

堆栈操作的原则

操作原则



先进后出



以字为单位

- 堆栈操作指令的操作数必为16位



堆栈指令对操作数的要求：

- 不能是立即数；
- 可以是16位寄存器或存储器1个字单元；
- 若为存储器操作数，需要声明操作数的字长。

堆栈操作指令

16位寄存器或存储器
字单元,
PUSH SS: [SP], OPRD



压栈指令 PUSH

· 格式: PUSH OPRD

16位寄存器或存储器
字单元,
POP OPRD, SS: [SP]



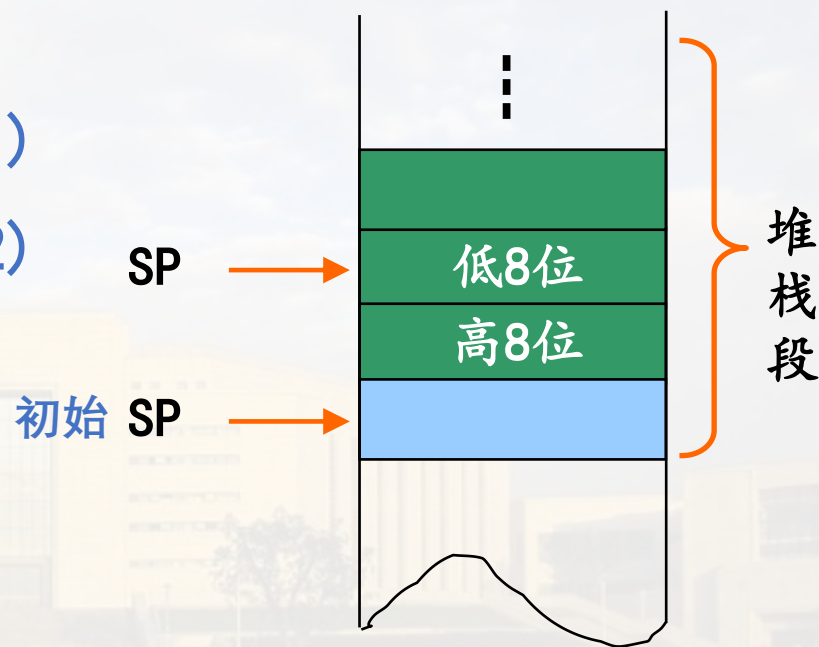
出栈指令 POP

· 格式: POP OPRD

压栈指令 PUSH

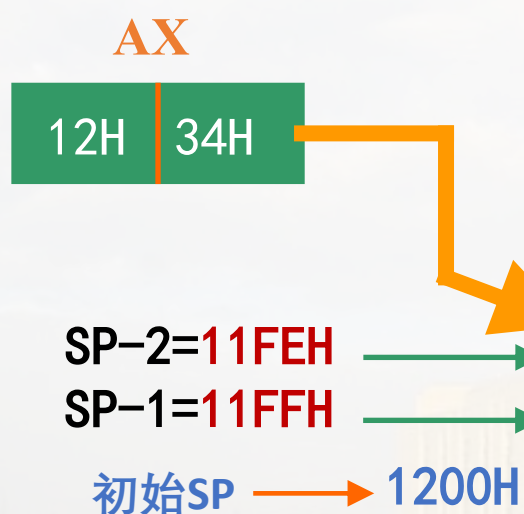
指令执行过程：先移动SP指针，后压数据

- $SP - 2 \rightarrow SP$
- 操作数高字节 $\rightarrow (SP-1)$
- 操作数低字节 $\rightarrow (SP-2)$



压栈指令的操作示例

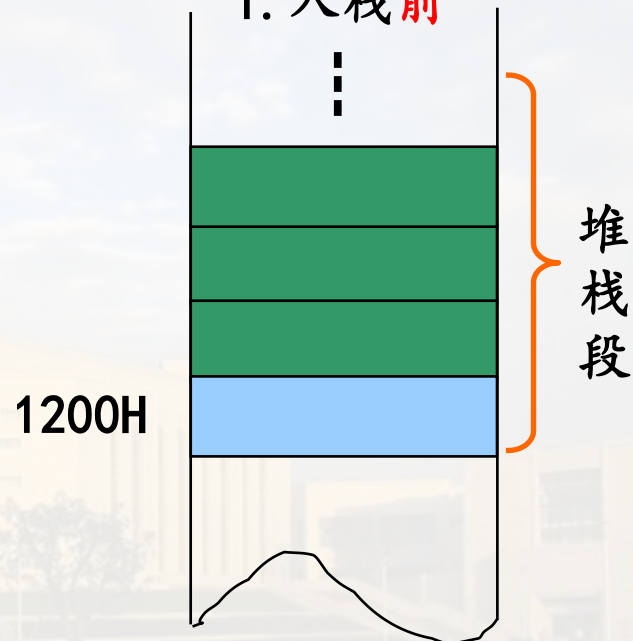
```
MOV  AX, 1234H  
MOV  SP, 1200H  
PUSH AX
```



2. 入栈后



1. 入栈前



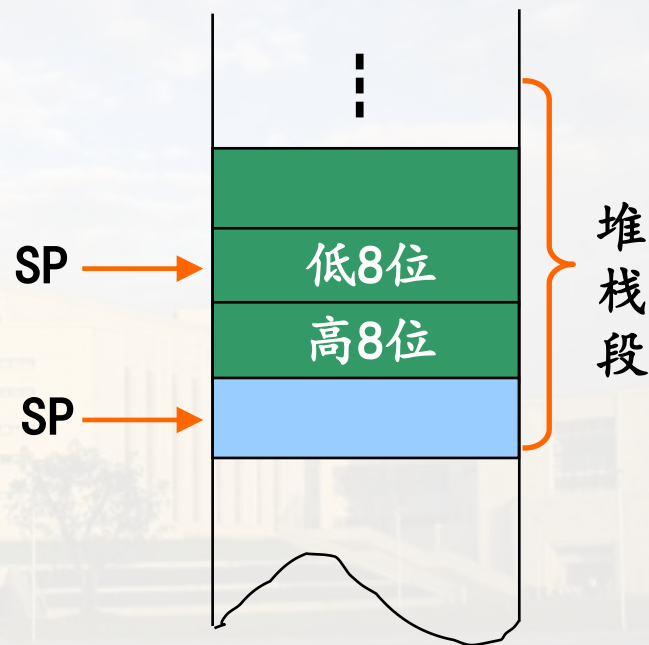
» 出栈指令POP

指令执行过程：先弹出数据，后移动SP指针

(SP) 操作数低字节 → 弹出

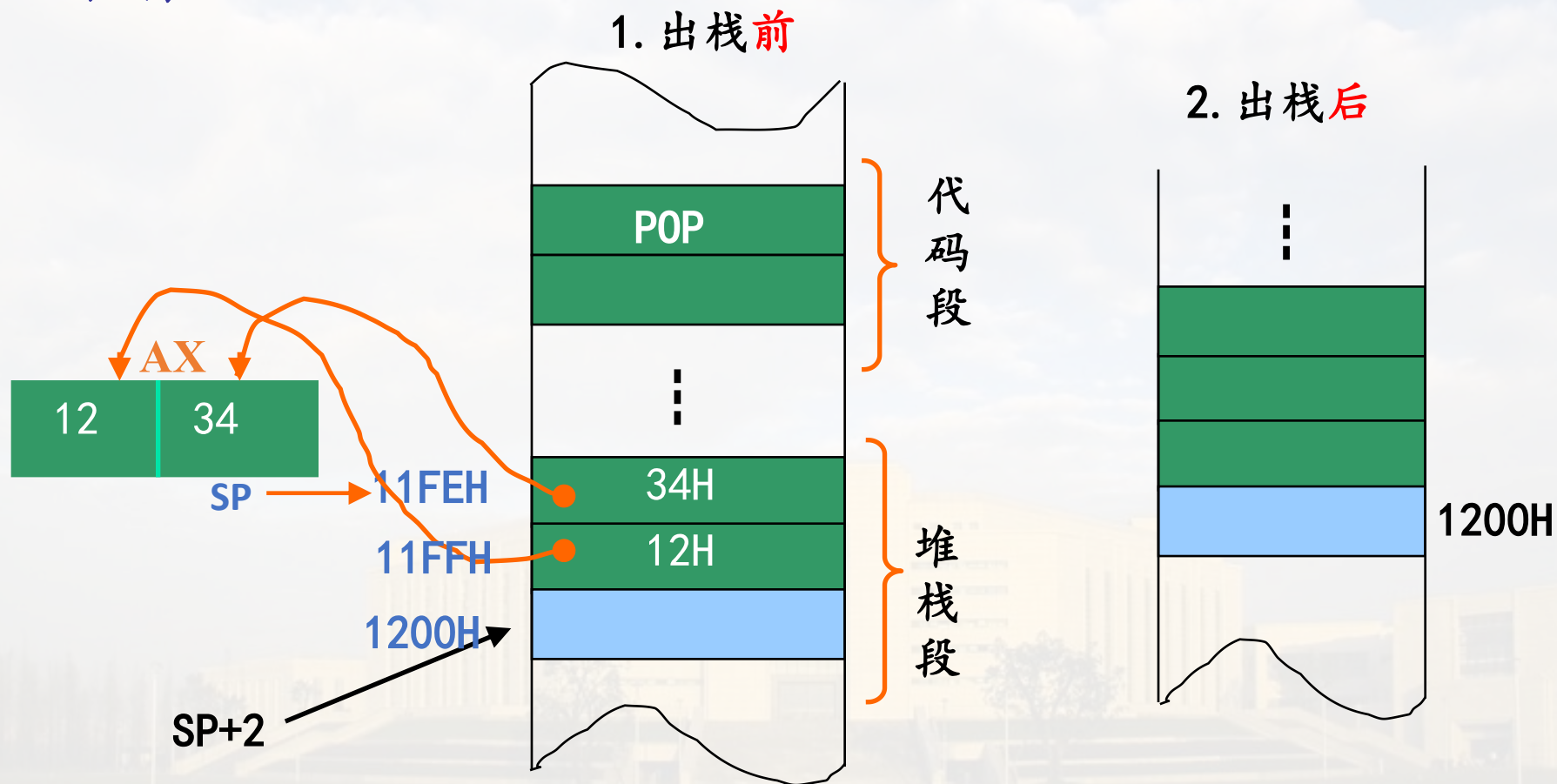
(SP+1) 操作数高字节 → 弹出

SP ← SP+2



出栈指令的操作示例

执行 POP AX



堆栈操作指令说明



01. 指令的操作数必须是16位；

02. 操作数可以是16位寄存器或存储器字单元，但不能是立即数；

不能通过出栈赋初值，但保护断点可以

03. 不能从栈顶弹出一个字给CS；

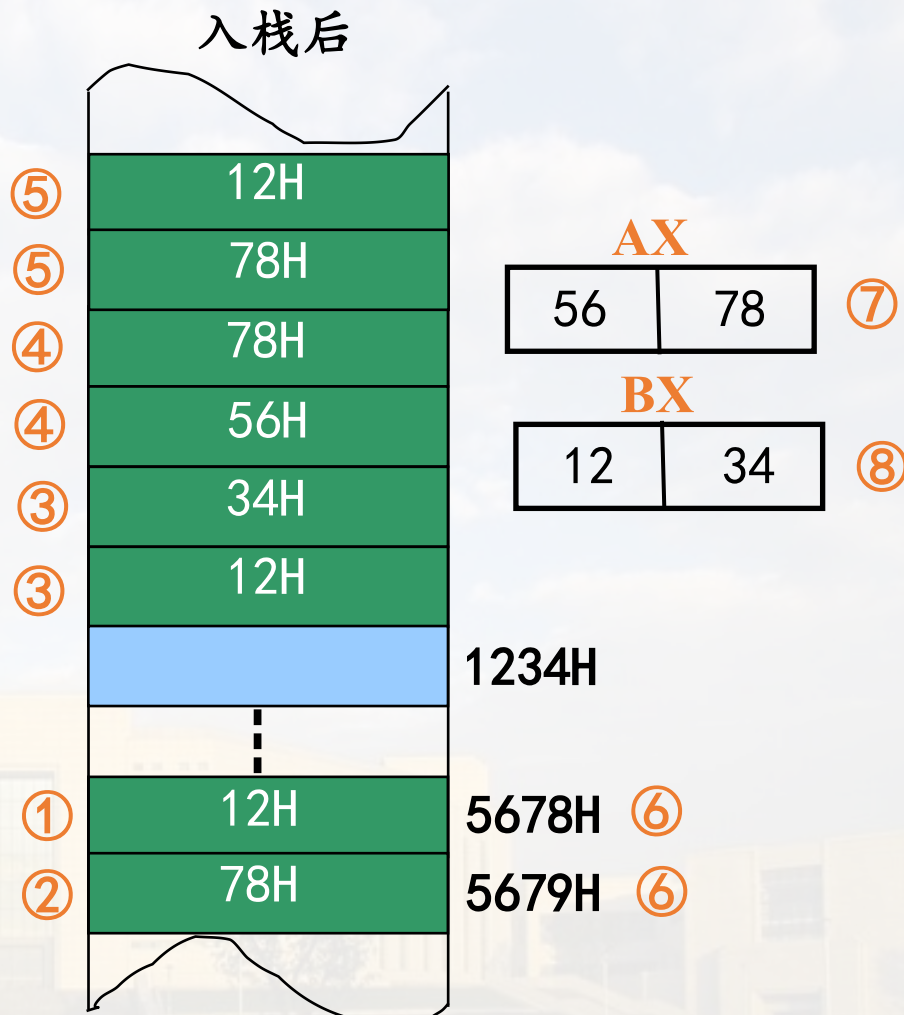
04. PUSH和POP指令在程序中一般成对出现；

05. PUSH指令的操作方向是从高地址向低地址，而POP指令的操作正好相反。

3.3 8086指令系统

堆栈操作指令例

- MOV AX, 1234H
- MOV SP, AX
- MOV BX, 5678H
- MOV [BX], AH ; ①
- MOV [BX+1], BL ; ②
- PUSH AX ; ③
- PUSH BX ; ④
- PUSH WORD PTR[BX] ; ⑤
- POP WORD PTR[BX] ; ⑥
- POP AX ; ⑦
- POP BX ; ⑧



如此，会使AX和BX的内容互换

» 交换指令

交换指令



格式:

- **XCHG** REG, MEM 或 MEM, REG 或 REG2, REG1

说明:

- 两操作数**必须有一个是寄存器操作数**
- 不允许使用段寄存器, **只能是通用寄存器**



例:

- XCHG AX, BX
- XCHG [2000H], CL



» 查表指令

查表指令



格式:

- **XLAT** ; 等价于 **XLAT AL, DS: [BX+AL]**

说明:

- 指令为**零操作数指令**, 采用**隐含寻址**, 隐含的操作数为**BX和AL**。
- 用**BX**的内容代表**表格首地址**, **AL**内容为**表内位移量**, **BX+AL**得到要查找元素的**偏移地址**

段: **DS**

BX: 距段首距离

AL: 表内位移量



操作:

- 将**BX+AL**所指单元的内容**送AL**

» 查表指令例

数据段中存放有一张ASCII码转换表，设首地址为2000H，现欲查出表中第11个代码的ASCII码，并将其值送AL。

2000H+0	30	'0'
	31	'1'
	32	'2'
	...	
2000H+11	39	'9'
	41	'A'
	42	'B'
	...	
	45	'E'
	46	'F'

» 查表指令例

可用如下指令实现:



```
MOV  BX, 2000H  
MOV  AL, 11  
XLAT
```

; BX ← 表首地址
; AL ← 序号
; 查表转换

执行后:



AL = 42H

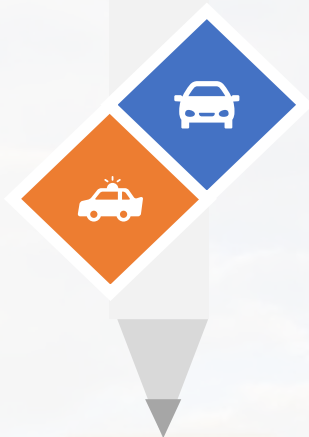
还可用其它方法实现, 如:



```
MOV  BX, 2000H  
MOV  AL, 0BH[BX]
```

» 字位扩展指令

将有符号数的符号位扩展到高位；



指令为零操作数指令，
采用隐含寻址，隐含的
操作数为AL或者AX与DX

CBW :AL → AX (即B → W)

CBD :AX → DX:AX (即W → DW)

字节到字的扩展指令



格式:

CBW

操作:

将AL的符号
位扩展到AH

规则:

- 若AL最高位=1, 则执行后AH=FFH
- 若AL最高位=0, 则执行后AH=00H

AL: 内容不变

» 字到双字的扩展指令



格式:

CWD

操作:

将AX符号位
扩展到DX

规则:

- 若AX最高位=1, 则
执行后DX=FFFFH
- 若AX最高位=0, 则
执行后DX=0000H

AX: 内容不变

3.3 8086指令系统

» 字位扩展指令例

判断以下指令执行结果：

MOV AL, 44H

CBW

MOV AX, 0AFDEH

CWD

MOV AL, 86H

CBW

指令执行结果：

AH=00H, AL=44H

DX=0FFFFH, AX=0AFDEH

AH=0FFH, AL=86H

» (专用) 输入输出指令

掌握

指令的格式及操作

指令的两种寻址方式

指令对操作数的要求

» 输入输出指令

输入输出指令

指令功能

指令格式

专门面向I/O端口操作的指令

- 输入指令：IN acc, PORT
- 输出指令：OUT PORT, acc
- PORT为端口地址，acc为累加器AX/AL

» 输入输出指令中寄存器特有的固定用法



累加器:所有I/O指令都通过AX/AL与接口传送信息,中间运算结果也多放于AX中;

In AX, 8位PORT
Out 8位PORT, AX



数据寄存器:在间接寻址的I/O指令中存放I/O端口的16位地址;在32位乘除法运算时,存放高16位数。

MOV DX, 0FC00H
In AX, DX

» 输入/输出指令寻址方式

根据端口地址码的长度，指令具有两种不同的端口地址表现形式。

直接寻址

- 端口地址为8位时，指令中直接给出8位端口地址；
- 可寻址256个端口。

$2^8=256$ ，即00H~FFH

间接寻址

- 端口地址为16位时，指令中的端口地址必须由DX指定；
- 可寻址64K个端口。

$2^{16}=64K$ ，即0100H~FFFFH

» 输入/输出指令格式:

输入指令格式:

a. 直接寻址

IN AX/AL, PORT (8位地址)

注: 端口地址 00H~FFH

b. 间接寻址

MOV DX, PORT (16位地址)

IN AX/AL, DX

注: 端口地址 0100H~FFFFH

输出指令格式:

a. 直接寻址

OUT PORT (8位地址), AX/AL

注: 端口地址 00H~FFH

b. 间接寻址

MOV DX, PORT (16位地址)

OUT DX, AX/AL

注: 端口地址 0100H~FFFFH

» I/O指令例

- 1. `IN AX/AL, 80H`; 直接寻址, 8位地址
- 2. `MOV DX, 2400H`; 间接寻址, 16位地址
■ `IN AX/AL, DX`; 输入一个字 (或字节)
- 3. `OUT 35H, AX/AL`; 直接寻址, 8位地址
- 4. `MOV DX, 2400H`; 间接寻址, 16位地址
■ `OUT DX, AX/AL`; 输出一个字 (或字节)

» 输入输出指令说明

01.

处理器一定是通过AL/AX与端口传送信息；

间接寻址的区别：

a. I/O指令间接寻址：

- `MOV DX, ****H` ; 16位地址
- `IN AX/AL, DX` ; (输入)
- `OUT DX, AX/AL` ; (输出)

b. 操作数寻址：

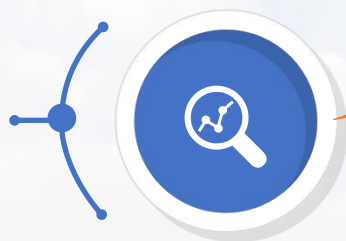
`MOV AX, [DX]` ; 寄存器间接

`MOV AX, [2000H]` ; 存储器直接

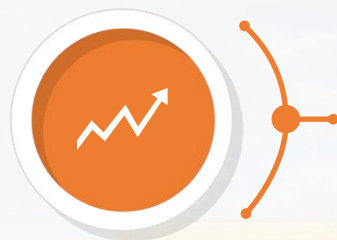


» 地址传送指令

取偏移地址指令LEA



取存储器16位偏移地址本身



取远地址指令：
LDS和LES

取存储器偏移地址所指向的存储器内连续4个单元内容作为段地址+偏移地址，即既有16位段地址（存入DS/ES），又有16位偏移地址（存入指令中指定16位寄存器），共取32位，4B

取偏移地址指令LEA

01

操作:

符号地址

将变量的16位偏移地址取出送目标寄存器

02

当程序中用符号地址表示内存偏移地址时，须使用该指令。

03

格式:

LEA REG, MEM

即直接、寄存器间接、寄存器相对、基址-变址、基址-变址-相对中之一

即BX、BP、SI、DI 中之一

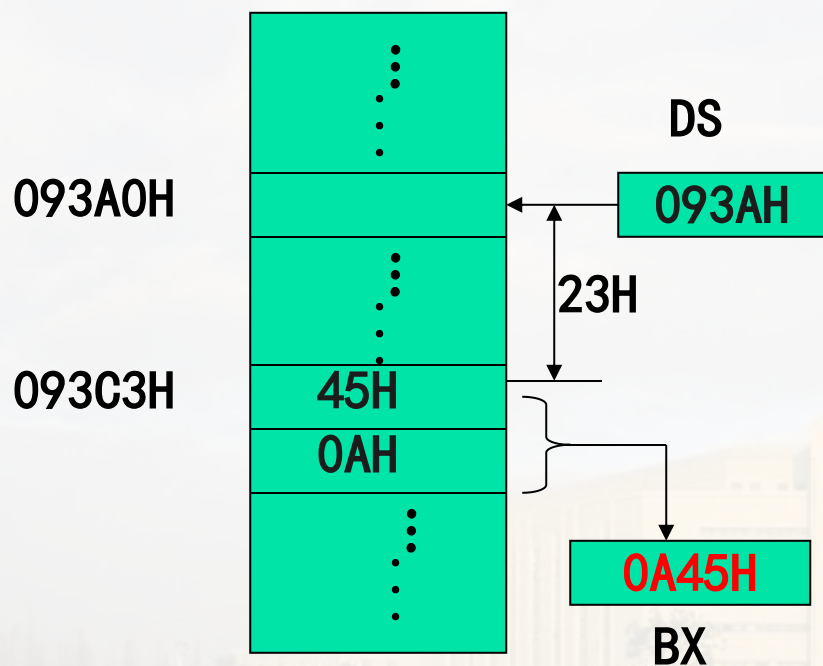
指令要求:

04

源操作数必须是一个存储器操作数，目标操作数必须是16位通用寄存器，通常是用作间址寄存器的寄存器。

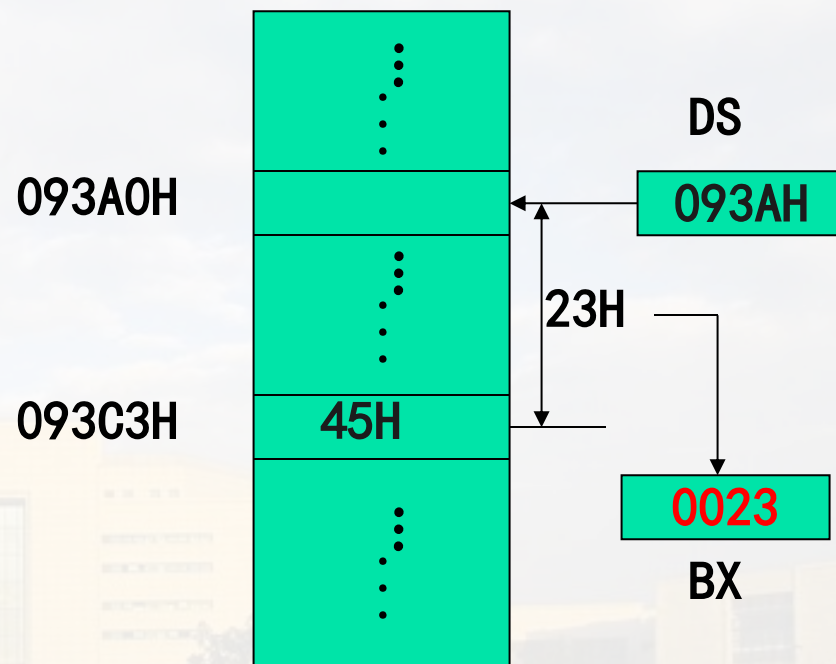
例：比较指令 `MOV BX, [23H]` 与 `LEA BX, [23H]`，
且 `DS=093AH`

`MOV BX, [23H]`



取偏移地址中的内容

`LEA BX, [23H]`



取偏移地址本身

» 又例如：

■ 比较下列指令：

1. MOV SI, DATA1
2. LEA SI, DATA1
3. MOV BX, [BX]
4. LEA BX, [BX]

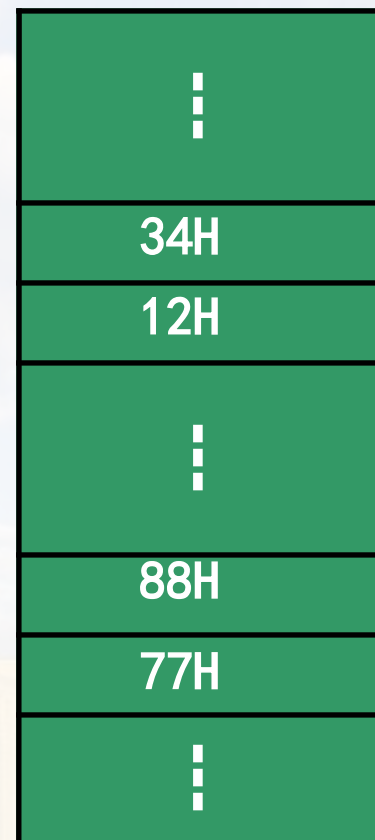
符号
地址

DATA1

上述4条指令执行后的结果？

1100H

BX=1100H



» 又例如：

■ 比较下列指令：

1. `MOV SI, DATA1`
2. `LEA SI, DATA1`
3. `MOV BX, [BX]`
4. `LEA BX, [BX]`

执行结果：SI=1234H

执行结果：SI=DATA1

执行结果：BX=7788H

执行结果：BX=1100H

符号
地址

DATA1

⋮
34H
12H
⋮
88H
77H
⋮

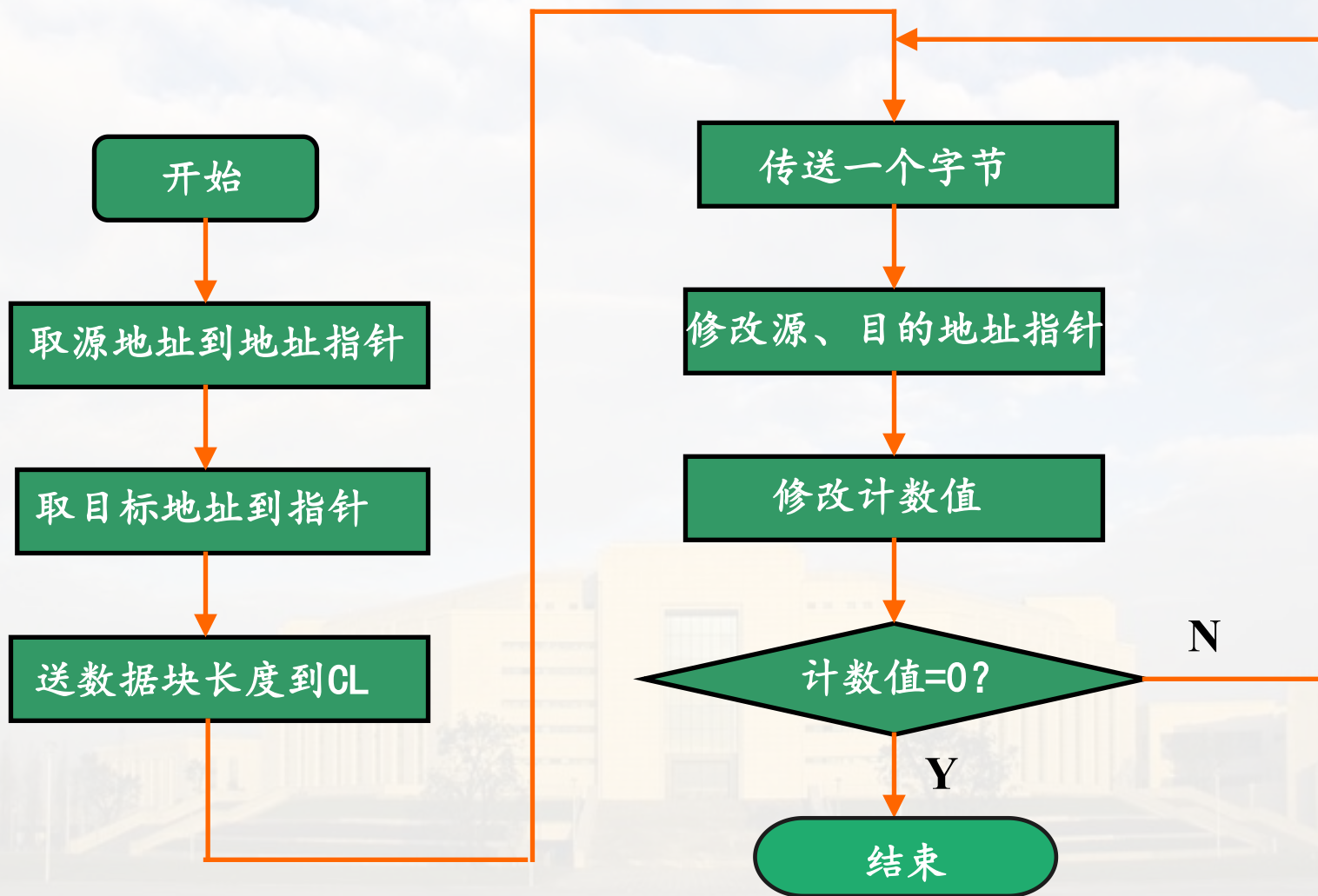
1100H

BX=1100H

» LEA指令在程序中的应用示例

将数据段中首地址为MEM1的50个字节的数据传送到同一逻辑段首地址为MEM2的区域存放。编写相应的程序段。（MEM1、MEM2块之间有间隔，且MEM2地址为高）

» LEA指令在程序中的应用示例



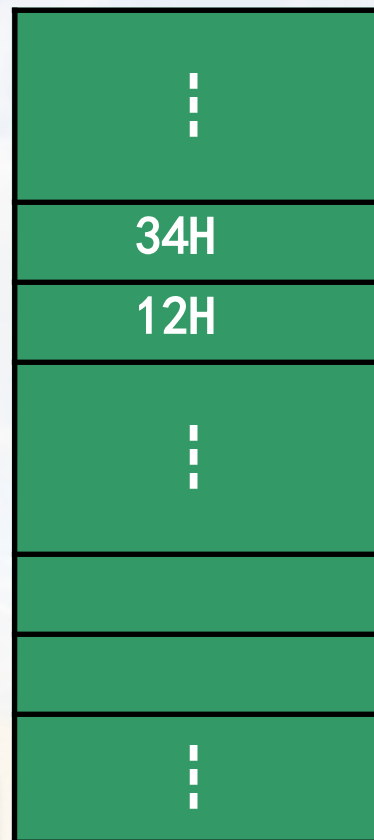
» LEA指令在程序中的应用示例

```
LEA    SI, MEM1
LEA    DI, MEM2
MOV    CL, 50
CLD
NEXT:  MOV    AL, [SI]
        MOV    [DI], AL
        INC    SI
        INC    DI
        DEC    CL
        JNZ    NEXT
HLT
```

MOV [DI], [SI]?

MEM1

MEM2



» 装入地址指针指令LDS和LES

格式:

LDS DEST, SRC

LES DEST, SRC



DEST 任意一个**16位**通用寄存器。



SRC 必须是一个**存储器操作数**。



作用

把SRC存储单元开始的**4个字节单元**的内容送入DEST通用寄存器和段寄存器DS (LDS指令) 或ES (LES指令)。



其它

低字单元内容为偏移量送**通用寄存器**，**高字单元**内容为段基值送**DS或ES**。

3.3 8086指令系统

装入地址指针指令LDS和LES

例：LDS SI, TABLE[BX]

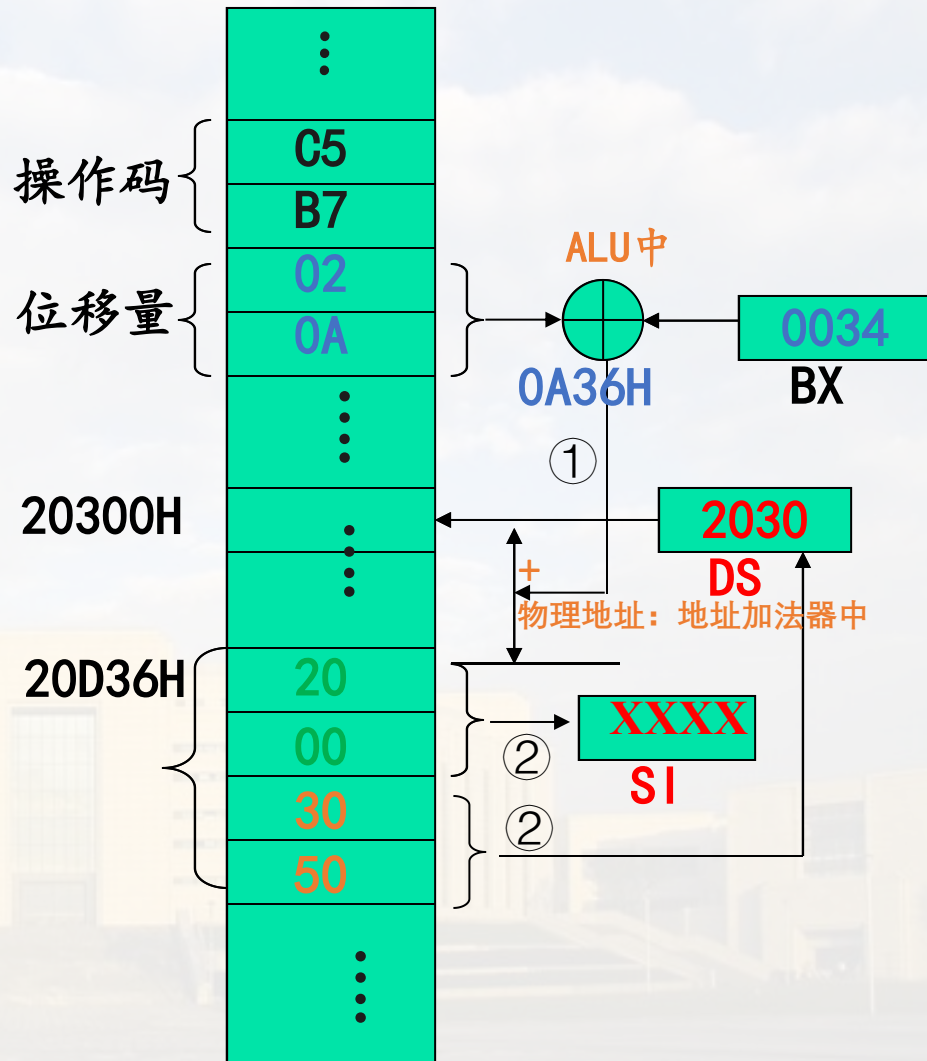
设 TABLE的值为0A02H,
(BX)=34H, (DS)=2030H

指令执行前：

注意DS、SI的
变化

DS段首物理地址：20300H

段内物理地址：20D36H



3.3 8086指令系统

装入地址指针指令LDS和LES

指令执行后：

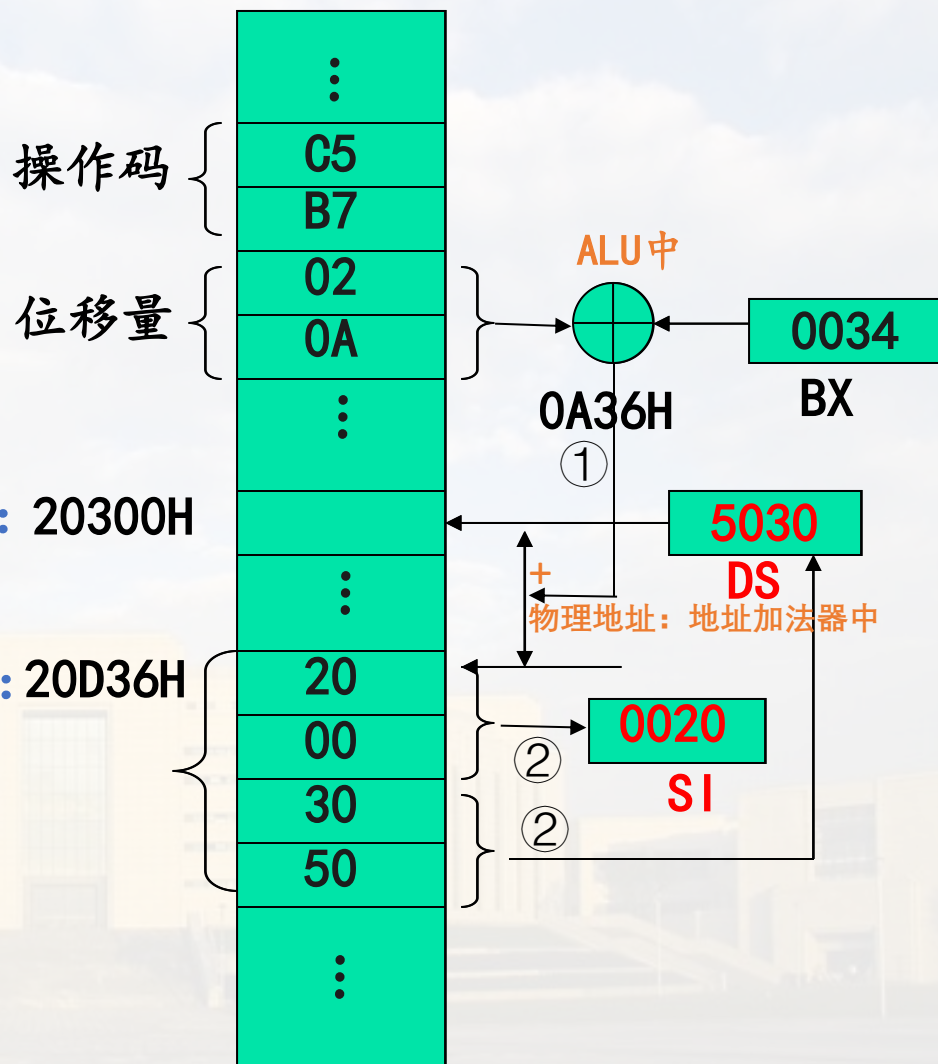
注意DS、SI的变化

DS: 2030H → 5030H

SI: XXXXH → 0020H

DS段首物理地址: 20300H

段内物理地址: 20D36H



» 标志位操作指令



LAHF



SAHF

LAHF AH , $FLAGS_L$
SAHF $FLAGS_L$, AH

隐含操作数AH和FLAGS



PUSHF



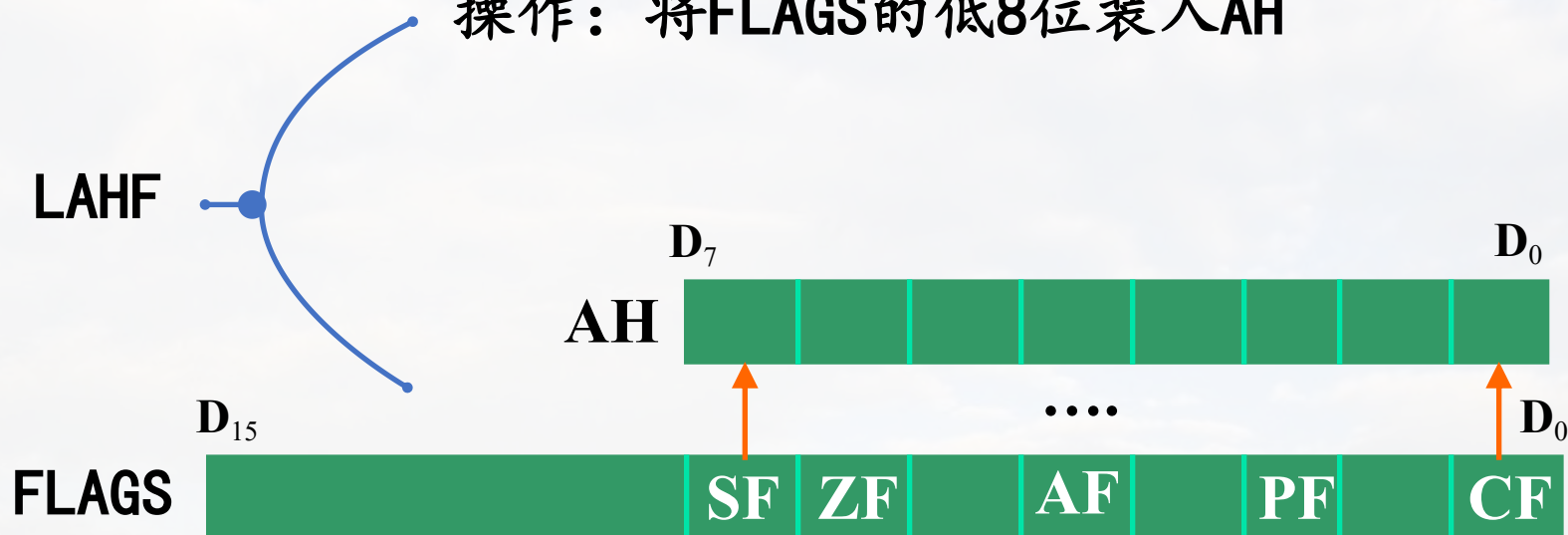
POPF

PUSHF SS:[SP] , $FLAGS$
POPF $FLAGS$, SS:[SP]

隐含操作数FLAGS和堆栈单元

» LAHF, SAHF

操作：将FLAGS的低8位装入AH



SAHF: 执行与LAHF相反的操作

» PUSHF, POPF

针对FLAGS的堆栈操作指令

- 将标志寄存器压栈或从堆栈弹出

PUSHF/POPF

» 算术运算类指令



算术运算指令的执行大多对状态标志位**会产生影响**(而传送类一般不会对FLAGS产生影响)

➤ 加法指令

01

普通加法指令ADD

02

带进位位的加法指令ADC

03

加1指令INC

加法
指令

加法指令对操作数的要求与MOV指令相同

» ADD指令



格式

ADD OPRD1, OPRD2



操作

OPRD1+OPRD2 $\longrightarrow$ OPRD1

ADD指令的执行对全部6个状态标志位都产生影响

» ADD指令例

MOV AL, 78H

ADD AL, 99H

ADD指令例

指令执行后6个状态标志位的状态

01111000

标志位状态: CF=1 SF=0

+ 10011001

AF=1 ZF=0

1

00010001

PF=1 OF=0

正加负不会产生溢出

» ADC指令



格式

指令格式、对操作数的要求、对标志位的影响与ADD指令完全一样



操作

$\text{OPRD1} + \text{OPRD2} + \text{CF} \longrightarrow \text{OPRD1}$

注意：CF是该指令执行前的值

ADC指令常用于多字节数相加，使用前要先将CF清零。使用CLC指令。

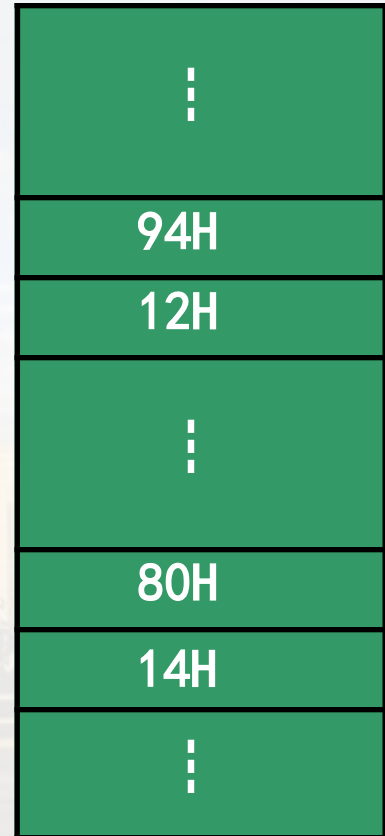
ADC指令应用例

求两个大数的和，两个数的长度都为20字节。放在M1, M2中，结果放M2中。

ADC指令应用例

```

LEA SI, M1
LEA DI, M2
MOV CX, 20
CLC          ; 使CF=0
NEXT : MOV AL, [SI]
      ADC [DI], AL
      INC SI
      INC DI
      DEC CX
      JNZ NEXT
      HLT
    
```



为什么要用
ADC?

» INC指令



格式

不能是段寄存器或立即数

INC OPRD ; 等价于 INC OPRD, 1



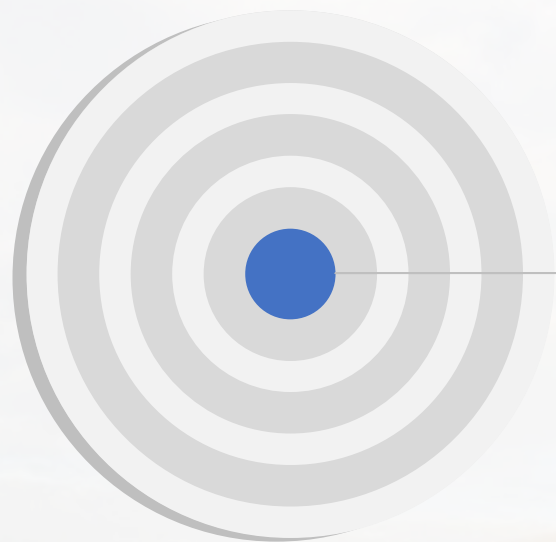
操作

$\text{OPRD} + 1 \longrightarrow \text{OPRD}$

注意：INC指令执行不影响CF标志

常用于在程序中修改地址指针

减法指令



01



普通减法指令SUB

02



带借位的减法指令SBB

03



减1指令DEC

04



比较指令CMP

05



求补指令NEG

减法指令对操作数的要求与对应的加法指令相同

» SUB指令



格式

SUB OPRD1, OPRD2



操作

OPRD1 - OPRD2 $\longrightarrow$ OPRD1

对标志位的影响与ADD指令同，影响6个状态位

» SBB指令



说明

指令格式、对操作数的要求、对标志位的影响与SUB指令完全一样



操作

$\text{OPRD1} - \text{OPRD2} - \text{CF} \longrightarrow \text{OPRD1}$

CF是该指令执行前的数值

» SBB指令应用例

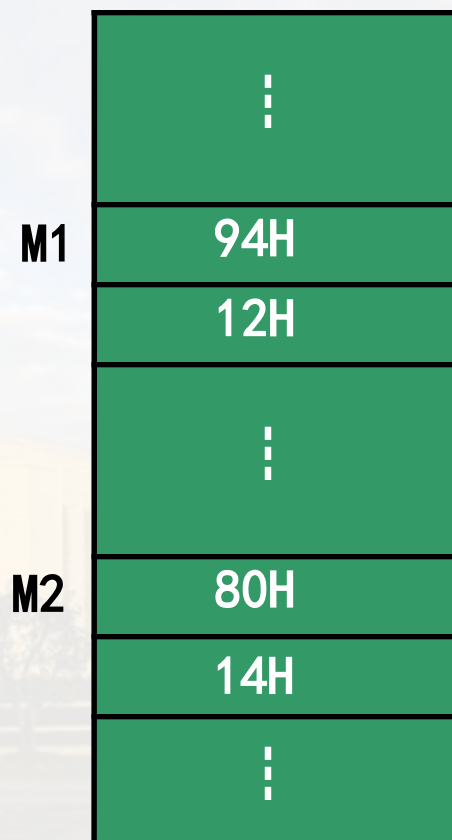
求两个大数的差，两个数的长度都为20字节。

减数、被减数放在M1, M2中，结果放M2中。

SBB指令应用例

```
LEA SI, M1
LEA DI, M2
MOV CX, 20
CLC          ; 使CF=0
NEXT : MOV AL, [SI]
      SBB [DI], AL
      INC SI
      INC DI
      DEC CX
      JNZ NEXT
      HLT
```

为什么要用
SBB?



» DEC指令



格式

DEC OPRD



操作

$\text{OPRD} - 1 \longrightarrow \text{OPRD}$

指令对操作数的要求与INC相同，不改变CF的值，
指令常用于在程序中修改计数值

» DEC指令例:实现一个计数循环程序

```
        MOV     AL, 10H
LOOP:   DEC     AL
        JNC     LOOP
```

DEC指令例

上述程序段中，是一个错误应用DEC指令的例子，错误地使用了CF标志位，**DEC指令不影响CF标志**。

应用程序示例：控制长时间延时

```
        MOV     BL, 2
NEXT1:  MOV     CX, 0FFFFH
NEXT2:  DEC     CX
        JNZ     NEXT2
        DEC     BL
        JNZ     NEXT1
        HLT
```

双重循环：内循环CX控制；外循环BL控制；延时程序

; ZF ≠ 0转NEXT2

; ZF ≠ 0转NEXT1

; 暂停执行

» NEG (求补) 指令



格式

NEG OPRD

8/16位寄存器或存储器操作数



操作

$0 - \text{OPRD} \longrightarrow \text{OPRD}$

用0减去操作数，相当于对该操作数求补，但不是补码。

所有位变反加1

» NEG (求补) 指令

求补运算:

所有位变反, 末位加1
(不分正负数)

补码运算:

正数: 符号位为0, 其余
位不变

负数: 符号位为1, 其余
位变反, 末位加1

对标志位的影响

对标志位影响

01

NEG指令将影响标志PF、AF、ZF、SF、CF和OF。

02

对进位标志CF的影响

- 只有当操作数为零时，进位标志CF被置零，其它情况都被置1

03

对溢出标志OF的影响

- 当字节操作数为-128 (80H)，或字操作数为-32768 (8000H) 时，结果将无变化，但溢出标志OF被置1

» CMP指令



格式

CMP OPRD1, OPRD2



操作

OPRD1- OPRD2

指令执行的结果不影响目标操作数，仅影响标志位！



用途

用于比较两个数的大小，可作为条件转移
指令转移的条件

指令对操作数的要求及对标志位的影响与SUB指令相同。

两个数大小的比较

CMP AX, BX

两个无符号数的比较

两个数的大小由CF或ZF来判断

若 $AX \geq BX$ CF=0

若 $AX < BX$ CF=1

若 $AX = BX$ ZF=1

另外，有专门的判断无符号数大小的指令：

JA/JAE/JB/JBE

两个数大小的比较

CMP AX, BX

两个带符号数的比较

两个数的大小由OF和SF共同决定

OF/SF同时=0, 或OF/SF同时=1

{ OF和SF状态相同
OF和SF状态不同

AX > BX

AX < BX

另外, 有专门的判断带符号数大小的指令:

JG/JGE/JL/JLE

» 程序功能

将放在BUF中的20个无符号数中找最大的数，并将其存放在MAX单元中。



» CMP指令例

```
LEA BX, MAX  
LEA SI, BUF  
MOV CL, 20  
MOV AL, [SI]
```

```
NEXT: INC SI
```

```
CMP AL, [SI]
```

```
JNC GOON
```

； CF=0转移，即 $AL > [SI]$

```
MOV AL, [SI]
```

； 即 $AL < [SI]$ ，更新AL

```
GOON: DEC CL  
JNZ NEXT  
MOV [BX], AL  
HLT
```

举例

乘法指令(自学, IMUL指令不要求)



无符号的乘法指令MUL



带符号的乘法指令IMUL

注意点:

- 乘法指令采用**隐含寻址**, 隐含的是存放被乘数的累加器**AL或AX**及存放结果的**AX, DX**;
- 若运算结果的高半部分是无效数值, 则 $OF=CF=0$, 否则 $OF=CF=1$ 。

» 无符号数乘法指令MUL

指令格式: **MUL OPRD**



不能是立即数，只能是通用R或M

操作:

OPRD为字节数 $\longrightarrow$ $AL \times OPRD \longrightarrow AX$

OPRD为字数 $\longrightarrow$ $AX \times OPRD \longrightarrow DX:AX$

➤ 无符号数乘法指令

指令格式：MUL OPRD



OPRD提供乘法运算的一个操作数，只能是寄存器或存储器操作数。



指令的另一操作数隐含使用AL或AX寄存器。



结果存放在AX(字节运算)或DX:AX(字乘法)中。

➤ 字节运算： $AX \leftarrow (AL) \times (OPRD)$

➤ 字运算： $DX:AX \leftarrow (AX) \times (OPRD)$

- ◆ MUL只对CF和OF标志产生有效影响，其他标志位的值不确定。
- ◆ 若结果的AH（字节运算）或DX（字运算）为全0，则CF=OF=0，否则CF=OF=1。

带符号数乘法指令IMUL

指令格式：IMUL OPRD



IMUL的功能除了操作数是带符号外，其余与MUL指令相同。

对标志位的影响：若乘积的高半部（AH或DX）是低半部的符号扩展（不是有效数值），则CF=OF=0。否则CF=OF=1。

» 举例

例如，对于字节乘法：

01

若乘积的 $(AH)=11111111$ ，且AL最高位为1，表示符号扩展，则 $CF=OF=0$ ；

02

若乘积的 $(AH)=00000000$ ，且AL最高为0，表示符号扩展，则 $CF=OF=0$ ；

03

若乘积的 $(AH)=11111111$ ，但AL最高位为0，不是符号扩展，则 $CF=OF=1$ ；

04

若乘积的 $(AH)=11111110$ ，不是符号扩展，则 $CF=OF=1$ ；

05

若乘积的 $(AH)=00000010$ ，不是符号扩展，则 $CF=OF=1$ 。

» 除法指令(自学, IDIV指令不要求)



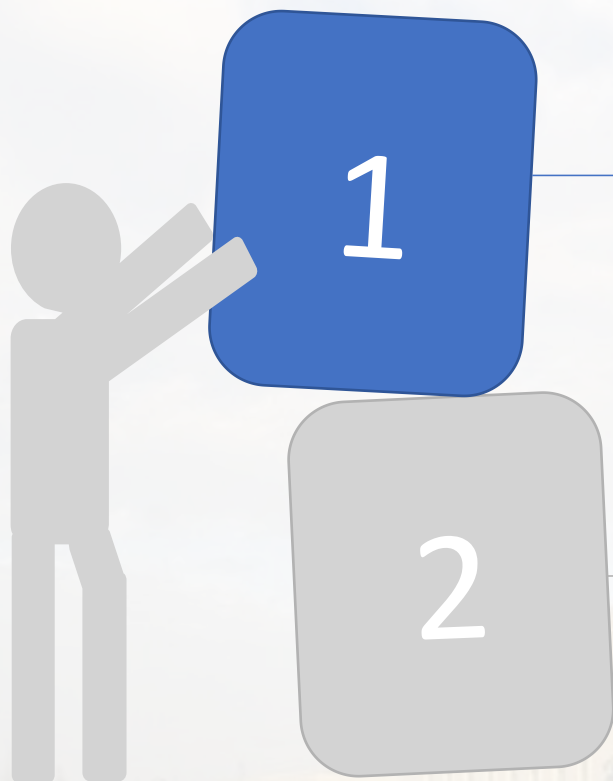
无符号除法指令
格式: DIV OPRD



有符号除法指令
格式: IDIV OPRD

除法指令的操作

指令要求被除数是除数的双倍字长



若OPRD是字节数

执行: $AX/OPRD$

结果: $AL=商$ $AH=余数$

若OPRD是双字节数

执行: $DX:AX/OPRD$

结果: $AX=商$ $DX=余数$

除法指令常与字节扩展CBW或字扩展指令CWD配合使用。

➤ 除法指令的操作（自学，DAA, AAA, DAS, AAS, AAM, AAD 指令不要求）



共6条（详见教材表3-3）：

DAA, AAA, DAS, AAS, AAM, AAD



都是隐含寻址方式，隐含的操作数是：

AL或者 AX（AL与AH）



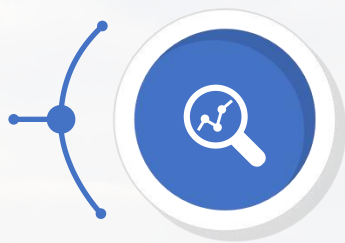
不能单独使用，要与相应的算术运算指令配合使用；加、减、乘、除指令。



执行结果为压缩BCD码或非压缩BCD码表示的十进制数。

» 十进制数的表示—BCD码（自学，BCD码指令不要求）

计算机中采用BCD码来表示十进制数。BCD码就是使用四位二进制数表示一位十进制数。

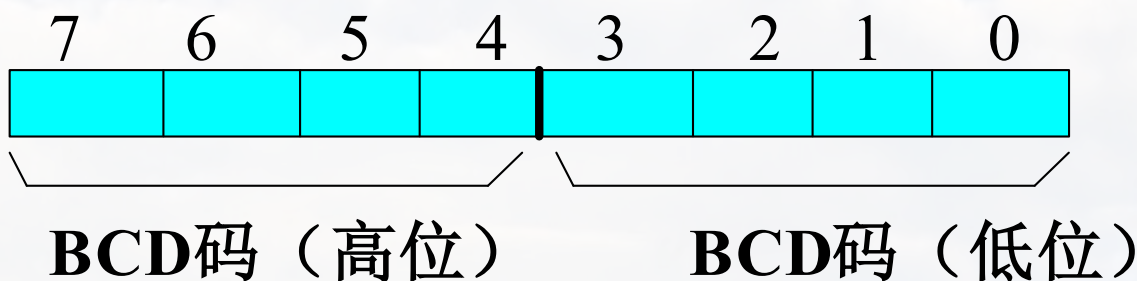


BCD码分为两种格式：

- 压缩型（组合型、装配型）
- 非压缩型（非组合型、拆散型）

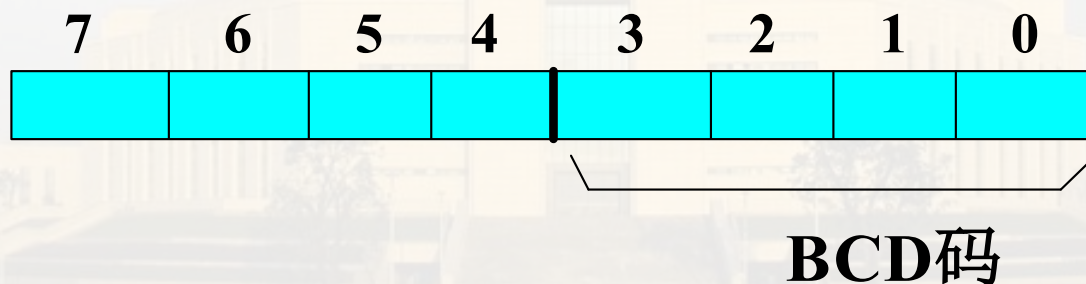
» 举例

压缩型：一个字节表示两个BCD码，即两位十进制数。



例如：**0010 0011** 表示十进制数的**23**

非压缩型：一个字节的低四位表示一个BCD码，而高四位对所表示的十进制数没有影响，常为0000或0011。



例如：**0000 1001**与**0011 1001**都是十进制数**9**的非压缩型BCD码

» 十进制数(BCD码)的运算有两种方法



数制转换:

先把十进制数转换为二进制数,再用二进制运算指令进行运算。最后将结果由二进制数转换为十进制数(BCD码)。



用BCD码处理指令:

直接进行十进制数运算。它又有两种方法:

- 指令系统提供专门实现BCD码运算的加、减、乘、除运算指令。
- 先用二进制数的加、减、乘、除运算指令对BCD码运算,再用BCD码调整指令对结果校正。--8086就是用这种

THANK

@刘辉

